유비샘 7차 세미나

https://github.com/ubisam-research/com.ubisam.boilerplate.frontend

https://github.com/ubisam-research/com.ubisam.boilerplate.backend

💡 이번 세미나는 해당 URL 을 참조하면 좋습니다.

BoilerPlate Front/Backend

Code Clone & Docker image

- 먼저 위 2개의 리포지토리를 클론해줍니다.

💡 git clone https://github.com/ubisam-research/com.ubisam.boilerplate.backend.git

```
> git clone https://github.com/ubisam-research/com.ubisam.boilerplate.backend.git
```

💡 git clone https://github.com/ubisam-research/com.ubisam.boilerplate.frontend.git

```
> git clone https://github.com/ubisam-research/com.ubisam.boilerplate.frontend.git
```

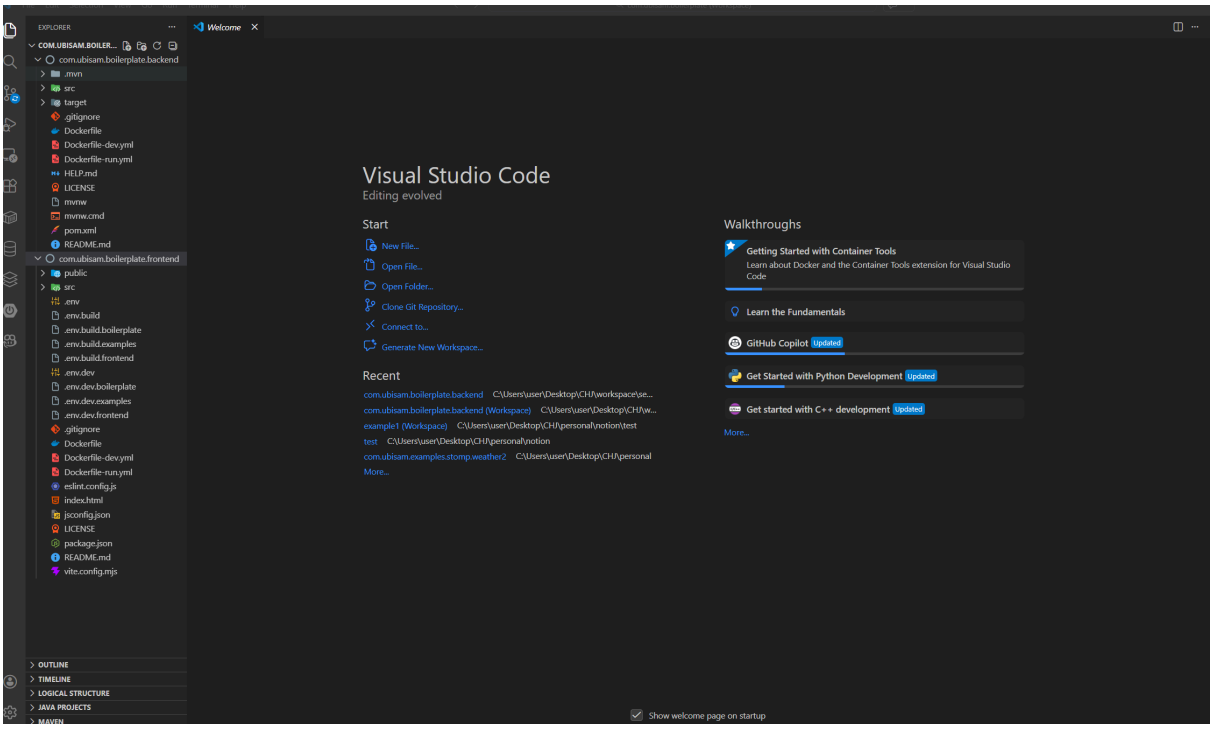
- 위 명령어를 입력해주시면 정상적으로 원하는 폴더에 clone 된 것을 확인해 보실 수 있습니다.

com.ubisam.boilerplate.backend	2026-03-25 오전 10:53	파일 폴더
com.ubisam.boilerplate.frontend	2026-03-25 오전 10:54	파일 폴더

- backend 나 frontend 둘 중 아무거나

  vscode 로 열어준 후 Workspace 추가를 통해 나머지 1개를 불러옵니다.

  아래 사진과 같이 불러온 것을 확인해 보실 수 있습니다.

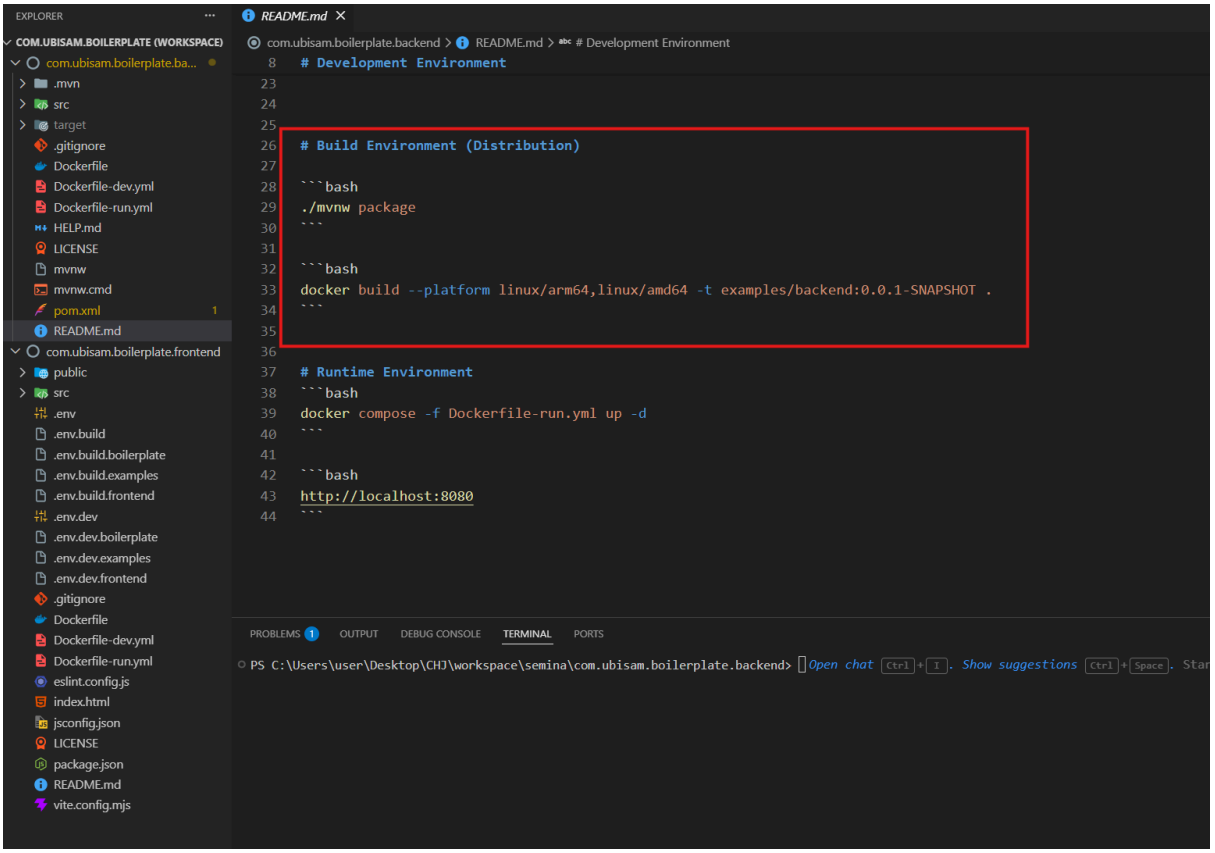


- 이제 Docker Image를 만들 차례입니다.

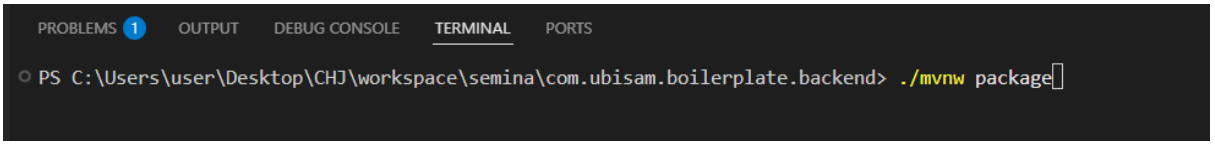
  Backend Image를 먼저 만들어보겠습니다.

- Backend의 <u>README.md</u> 로 가줍니다.

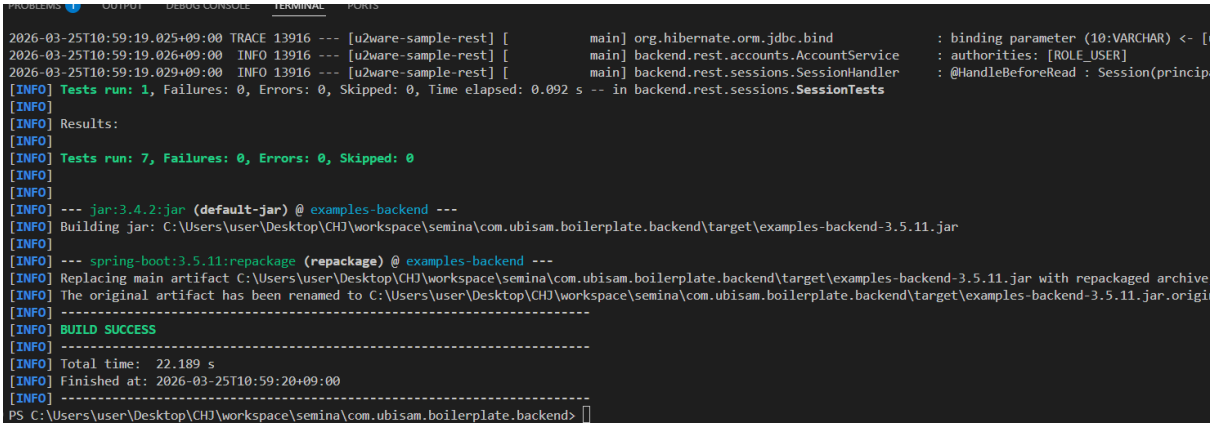
  아래 사진의 명령어를 backend를 클론한 곳에서 터미널로 입력해줍니다.



./mvnw package

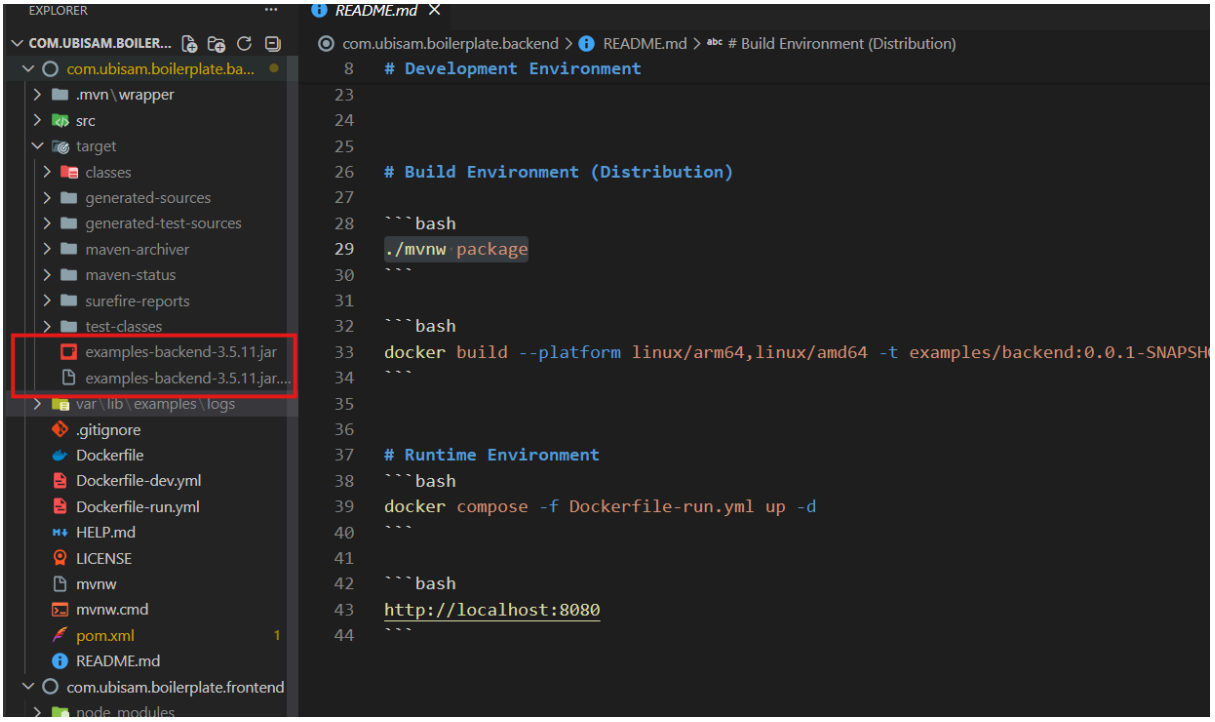


- 아래 사진과 같이 BUILD SUCCESS가 뜨면 성공입니다.



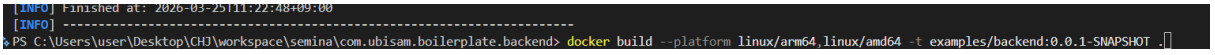
> 만약 빨간색(Build Failure)이 뜨고 빌드에 실패할 경우
> 에 해결방안을 추가해놨습니다.

- 정상적으로 작동했다면 target 폴더 밑에 .jar 파일들이 추가된 것을 확인해 보실 수 있습니다.



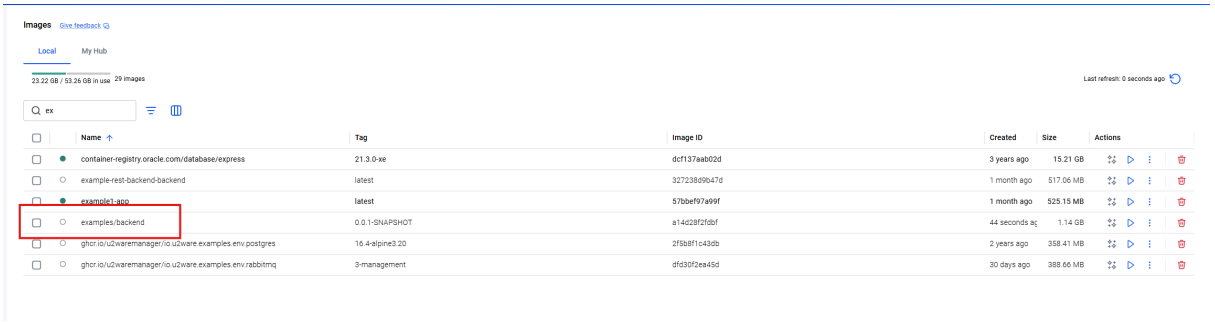
- 아래 명령어를 입력해줍니다.

> docker build --platform linux/arm64,linux/amd64 -t examples/backend:0.0.1-SNAPSHOT .



- 아래 사진과 같이 Docker Image가 만들어진 것을 확인해 보실 수 있습니다.

```
PS C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.backend> docker build --platform linux/arm64,linux/amd64 -t examples/backend:0.0.1-SNAPSHOT .
[+] Building 7.7s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 263B
 => [linux/arm64 internal] load metadata for docker.io/library/openjdk:24-ea-17-jdk-slim-bullseye
 => [linux/amd64 internal] load metadata for docker.io/library/openjdk:24-ea-17-jdk-slim-bullseye
 => [auth] library/openjdk:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 2B
 => CACHED [linux/amd64 1/2] FROM docker.io/library/openjdk:24-ea-17-jdk-slim-bullseye@sha256:d3d89f0ff751bd92d856f11a0b3223744fbe55ee95ba8990bcfe016223762079
 => => resolve docker.io/library/openjdk:24-ea-17-jdk-slim-bullseye@sha256:d3d89f0ff751bd92d856f11a0b3223744fbe55ee95ba8990bcfe016223762079
 => CACHED [linux/arm64 1/2] FROM docker.io/library/openjdk:24-ea-17-jdk-slim-bullseye@sha256:d3d89f0ff751bd92d856f11a0b3223744fbe55ee95ba8990bcfe016223762079
 => => resolve docker.io/library/openjdk:24-ea-17-jdk-slim-bullseye@sha256:d3d89f0ff751bd92d856f11a0b3223744fbe55ee95ba8990bcfe016223762079
 => [internal] load build context
 => => transferring context: 69.84MB
 => [linux/arm64 2/2] COPY ../target/*.jar /var/lib/examples/backend/app.jar
 => [linux/amd64 2/2] COPY ../target/*.jar /var/lib/examples/backend/app.jar
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:2fd45677677201cc598c17b5a2ce92b2b434448ae207ea82363157b809fe224d
 => => exporting config sha256:f85018cd230cdd34947fe4a1066dec914dbcb64b6ab24d334864c9fe3460ff60
 => => exporting attestation manifest sha256:a5c39ea8311215a2d41f1e1eb23c9721a795e4ddd13853d7360d7a1a0d304cf9
 => => exporting manifest sha256:5fda477a3ed192957a7e474d770eb8fc8acade724a12237231e89a39447d98c8
 => => exporting config sha256:e2992d131d321d9f9c80a09411d1fe44e4c0ef8d32f743a8ace5abd929f523c3
 => => exporting attestation manifest sha256:7e0fc6ad78c3bd322802263c62fb67bfedfc91cf274112df62beacff83ba8b9a
 => => exporting manifest list sha256:a14d28f2fdbf5a97bf3d44fb3a37052a620a0a5f6367425d59237c6b5e38437a
 => => naming to docker.io/examples/backend:0.0.1-SNAPSHOT
 => => unpacking to docker.io/examples/backend:0.0.1-SNAPSHOT
PS C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.backend>
```



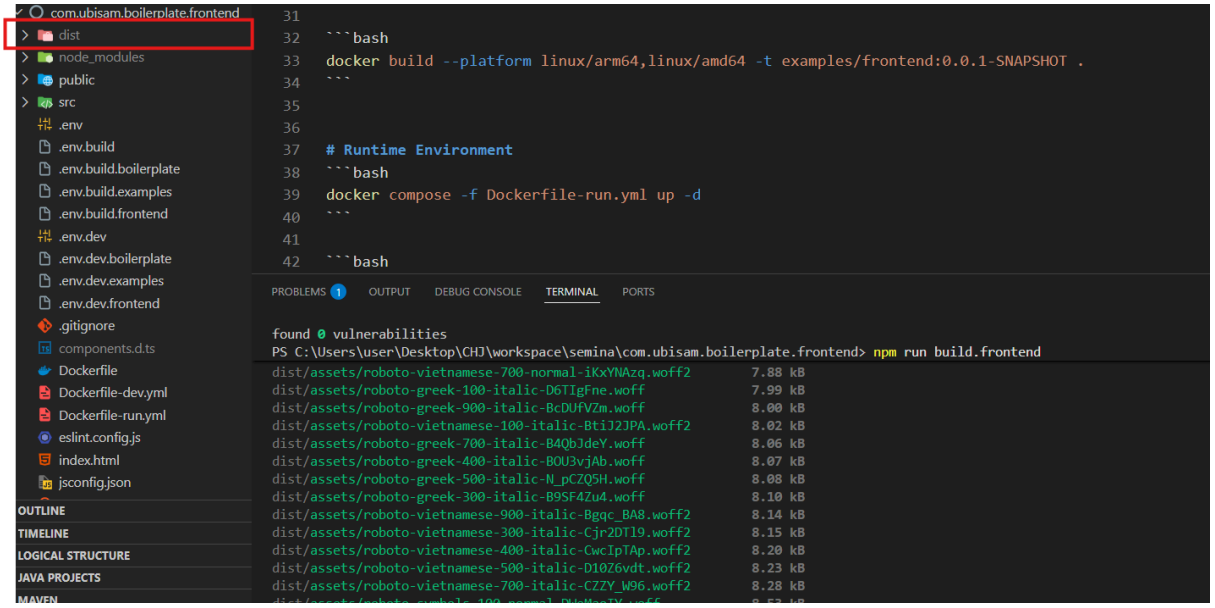
- 이제 Frontend 터미널로 이동해줍니다. 그 후 Frontend의 README.md를 열어줍니다.
- 아래 사진의 3개의 명령어를 입력해줍니다.

```
7
8   # Development Environment
9   ```bash
10  docker compose -f Dockerfile-dev.yml up -d
11  ```
12
13  ```bash
14  npm install
15  ```
16
17  ```bash
18  npm run dev.frontend
19  ```
20
21  ```bash
22  http://localhost:3000
23  ```
24
25
26  # Build Environment (Distribution)
27
28  ```bash
29  npm run build.frontend
30  ```
31
32  ```bash
33  docker build --platform linux/arm64,linux/amd64 -t examples/frontend:0.0.1-SNAPSHOT .
34  ```
```

💡 npm install

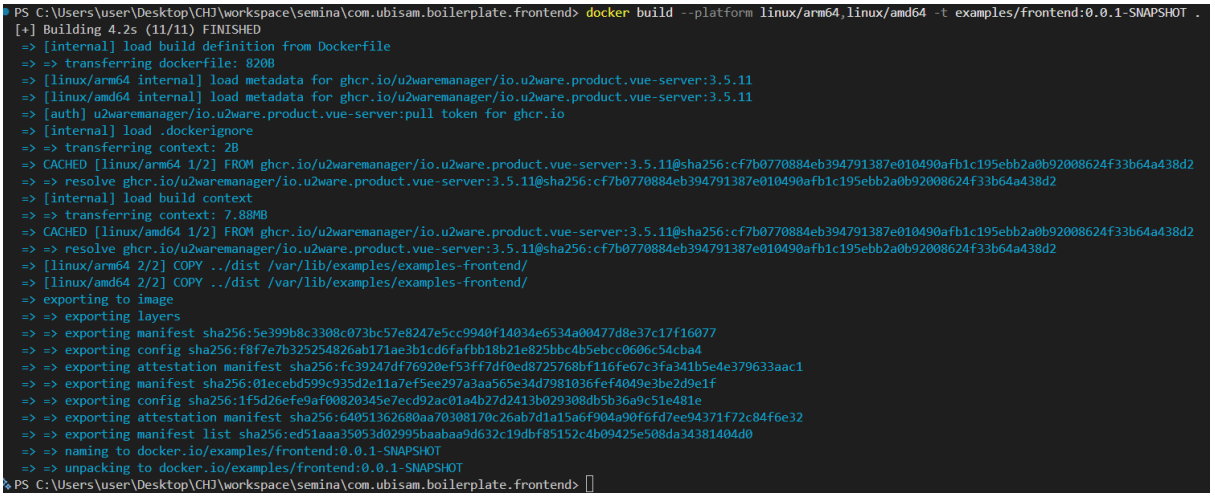
```
PS C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.frontend> npm install

up to date, audited 374 packages in 960ms

112 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.frontend>
```

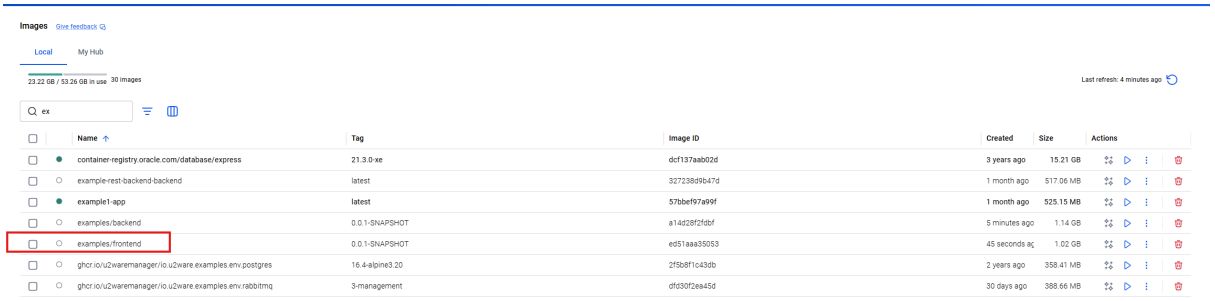
💡 npm run build.frontend
해당 명령어 입력 후 dist 폴더가 생긴 것을 확인해 보실 수 있습니다.



> docker build --platform linux/arm64,linux/amd64 -t examples/frontend:0.0.1-SNAPSHOT .

- 아래 사진과 같이 Docker Image가 만들어진 것을 확인해 보실 수 있습니다.

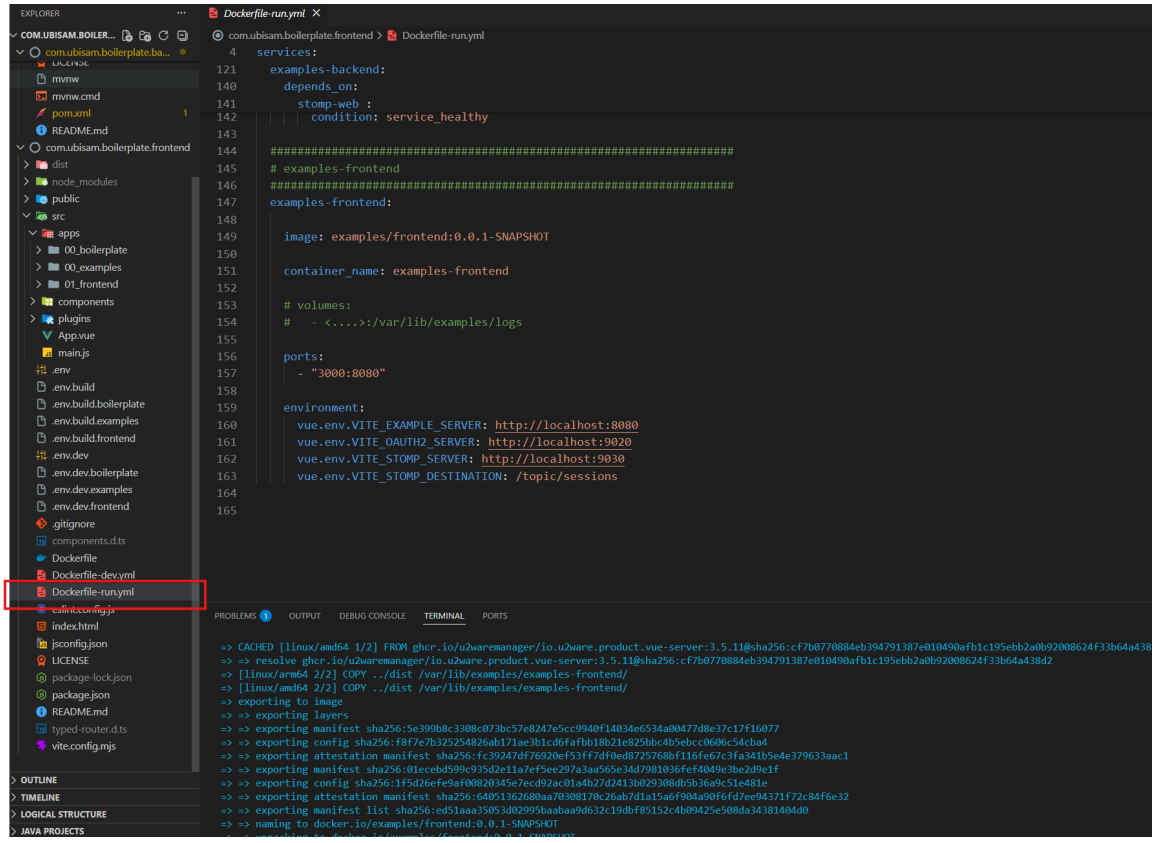




- 최종적으로 위와 같이 저희 제품에 대한 Docker image 가 나오는 것을 확인해 보실 수 있습니다.

Docker Container

- Frontend 코드에 Dockerfile-run.yml 파일로 이동합니다.



  - oauth2-db는 인증서버의 DB서버입니다.

```yaml
#####################################################################
# oauth2-db
#####################################################################
oauth2-db:

  image: ghcr.io/u2waremanager/io.u2ware.examples.env.postgres:16.4-alpine3.20

  container_name: oauth2-db

  ports:
    - "5432:5432"

  environment:
    POSTGRES_HOST_AUTH_METHOD: trust
    POSTGRES_USER: oauth2-db
    POSTGRES_PASSWORD: oauth2-db
    POSTGRES_DB: oauth2-db
```

- oauth2-web은 인증서버 클라이언트입니다.

```yaml
#####################################################################
# oauth2-web
#####################################################################
oauth2-web:
  image: ghcr.io/u2waremanager/io.u2ware.product.oauth2-server:3.5.11

  container_name: oauth2-web

  # volumes:
  #   - <....>:/var/lib/u2ware/logs

  ports:
    - "9020:9020"

  environment:
    server.port: 9020
    spring.security.user.name: oauth2-web
    spring.security.user.password: oauth2-web

    spring.jpa.hibernate.ddl-auto: update
    spring.jpa.properties.hibernate.dialect: org.hibernate.dialect.PostgreSQLDialect

    spring.datasource.driver-class-name: org.postgresql.Driver
    spring.datasource.url: jdbc:postgresql://oauth2-db:5432/oauth2-db
    spring.datasource.username: oauth2-db
    spring.datasource.password: oauth2-db

    io.u2ware.common.oauth2.clients.example-backend: http://examples-backend:8080/oauth2/token

  healthcheck:
    test: ["CMD", "ls", "-al"]
    interval: 60s
    timeout: 3s
    retries: 1
    start_period: 3s
```

- stomp-mq는 rabbitmq 설정입니다.

```yaml
#####################################################################
# stomp-mq
#####################################################################
stomp-mq:
  image: ghcr.io/u2waremanager/io.u2ware.examples.env.rabbitmq:3-management

  container_name: stomp-mq

  ports:
    - "15672:15672"
    - "5672:5672"
    - "61613:61613"

  environment:
    RABBITMQ_DEFAULT_USER: stomp-mq
    RABBITMQ_DEFAULT_PASS: stomp-mq
```

- stomp-server는 stomp 클라이언트 입니다.

```yaml
#######################################################################
# stomp-server
#######################################################################
stomp-web:
  image: ghcr.io/u2waremanager/io.u2ware.product.stomp-server:3.5.11

  container_name: stomp-web

  # volumes:
  #   - <....>:/var/lib/u2ware/logs

  ports:
    - "9030:8080"

  environment:
    spring.security.oauth2.resourceserver.jwt.issuer-uri: http://localhost:9020
    spring.security.oauth2.resourceserver.jwt.jwk-set-uri: http://oauth2-web:9020/oauth2/jwks

    io.u2ware.common.stomp.publishing: /topic/sessions

    io.u2ware.common.stomp.broker.host: stomp-mq
    io.u2ware.common.stomp.broker.port: 61613
    io.u2ware.common.stomp.broker.username: stomp-mq
    io.u2ware.common.stomp.broker.password: stomp-mq

  depends_on:
    oauth2-web :
      condition: service_healthy

  healthcheck:
    test: ["CMD", "ls", "-al"]
    interval: 60s
    timeout: 3s
    retries: 1
    start_period: 3s
```

- examples-backend는 방금 만든 backend 입니다.

```yaml
#################################################################
# examples-backend
#################################################################
examples-backend:
  image: examples/backend:0.0.1-SNAPSHOT

  container_name: examples-backend

  # volumes:
  #   - <....>:/var/lib/examples/logs

  ports:
    - "8080:8080"

  environment:
    spring.security.oauth2.resourceserver.jwt.issuer-uri: http://localhost:9020
    spring.security.oauth2.resourceserver.jwt.jwk-set-uri: http://oauth2-web:9020/oauth2/jwks

    io.u2ware.common.stomp.url: ws://stomp-web:8080/stomp
    io.u2ware.common.stomp.subscriptions.sessions: /topic/sessions
    io.u2ware.common.stomp.subscriptions.others: /topic/others

  depends_on:
    stomp-web :
      condition: service_healthy
```

- examples-frontend 는 방금 만든 frontend 입니다.

```yaml
#################################################################
# examples-frontend
#################################################################
examples-frontend:

  image: examples/frontend:0.0.1-SNAPSHOT

  container_name: examples-frontend

  # volumes:
  #   - <....>:/var/lib/examples/logs

  ports:
    - "3000:8080"

  environment:
    vue.env.VITE_EXAMPLE_SERVER: http://localhost:8080
    vue.env.VITE_OAUTH2_SERVER: http://localhost:9020
    vue.env.VITE_STOMP_SERVER: http://localhost:9030
    vue.env.VITE_STOMP_DESTINATION: /topic/sessions
```

- Frontend 코드의 README.md로 이동합니다.

그 후 맨 마지막에 Runtime Environment로 이동합니다.

아래 사진에 표시된 명령어를 Frontend 터미널에서 실행합니다.

💡 docker compose -f Dockerfile-run.yml up -d

```
# Build Environment (Distribution)

```bash
npm run build.frontend
```

```bash
docker build --platform linux/arm64,linux/amd64 -t examples/frontend:0.0.1-SNAPSHOT .
```

Runtime Environment
```bash
docker compose -f Dockerfile-run.yml up -d
```

```bash
http://localhost:3000
```
```

```
PS C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.frontend> docker compose -f Dockerfile-run.yml up -d
```

- 실행 시 아래 사진과 같이 총 6개가 보이고 Docker Desktop으로 확인하시면 examples 컨테이너가 생기고 6개의 컨테이너가 묶인 것을 확인해 보실 수 있습니다.

```
PS C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.frontend> docker compose -f Dockerfile-run.yml up -d
[+] up 7/7
 ✓ Network examples_default    Created
 ✓ Container stomp-mq          Started
 ✓ Container oauth2-db         Started
 ✓ Container oauth2-web        Healthy
 ✓ Container examples-frontend Started
 ✓ Container stomp-web         Healthy
 ✓ Container examples-backend  Started
PS C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.frontend>
```

examples	-	-	-	1.67%	1.64GB / 45.69GI	21.53%	329.2MB / 54.49	77.77KB / 66.02	53, 87, 48, 56, 16,	1 minute ago			
stomp-web	43fd8ecb1271	u2waremanager/io.u2ware.product.stomp-server:3.5.11	9030:8080	0.3%	387.4MB / 7.61G	4.97%	41.7MB / 340KB	15.3KB / 9.7KB	87	1 minute ago			
stomp-mq	3e062e752f98	u2waremanager/io.u2ware.examples.env.rabbitmq:3-ma	15672:15672 show all ports (3)	0.24%	167.6MB / 7.61G	2.15%	61MB / 606KB	4KB / 1.5KB	56	1 minute ago			
oauth2-web	8cecfba158b5	u2waremanager/io.u2ware.product.oauth2-server:3.5.11	9020:9020	0.47%	408.9MB / 7.61G	5.24%	85MB / 90.1KB	20.8KB / 23.8KB	57	1 minute ago			
oauth2-db	cddc549e3789	u2waremanager/io.u2ware.examples.env.postgres:16.4-	5432:5432	0.06%	71.12MB / 7.61G	0.91%	38.1MB / 53.3ME	22KB / 16.1KB	16	1 minute ago			
frontend	fed5ff1f830e	examples/frontend:0.0.1-SNAPSHOT	3000:8080	0.35%	203.8MB / 7.61G	2.61%	49.3MB / 77.8KB	2.67KB / 126B	48	1 minute ago			
backend	14c219ed79ff	examples/backend:0.0.1-SNAPSHOT	8080:8080	0.25%	440.3MB / 7.61G	5.65%	54.1MB / 106KB	13KB / 14.8KB	53	1 minute ago			

화면 테스트

- 해당 Docker Container가 정상적으로 열린 것이 확인되면 localhost:3000 으로 접속합니다.

- 다음으로 다른 탭에 oauth2-web으로 접속합니다.

  Frontend의 Dockerfile-run.yml에 가서 포트를 확인합니다.

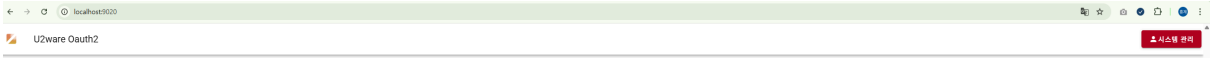
```
#####################################################################
# oauth2-web
#####################################################################
oauth2-web:
  image: ghcr.io/u2waremanager/io.u2ware.product.oauth2-server:3.5.11

  container_name: oauth2-web

  # volumes:
  #   - <....>:/var/lib/u2ware/logs

  ports:
    - "9020:9020"
```

- localhost:9020 으로 접속합니다.

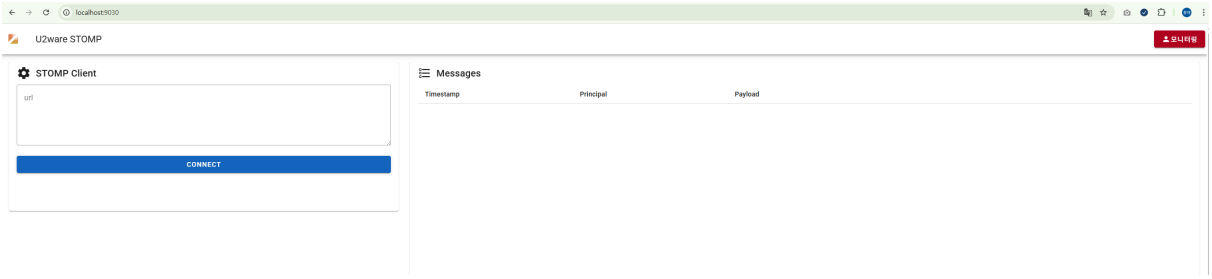


- 다음으론 stomp-server로 접속합니다.

  Frontend의 Dockerfile-run.yml에 가서 포트를 확인합니다.

```yaml
##################################################################
# stomp-server
##################################################################
stomp-web:
  image: ghcr.io/u2waremanager/io.u2ware.product.stomp-server:3.5.11

  container_name: stomp-web

  # volumes:
  #   - <....>:/var/lib/u2ware/logs

  ports:
    - "9030:8080"
```

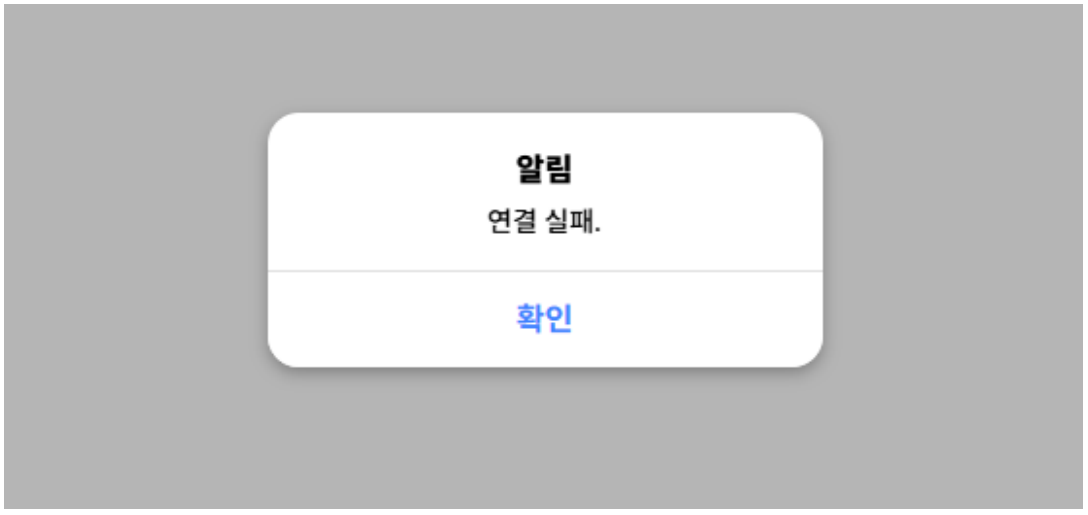
- localhost:9030 으로 접속합니다.



- 먼저 STOMP Client 연결을 시도해보겠습니다.

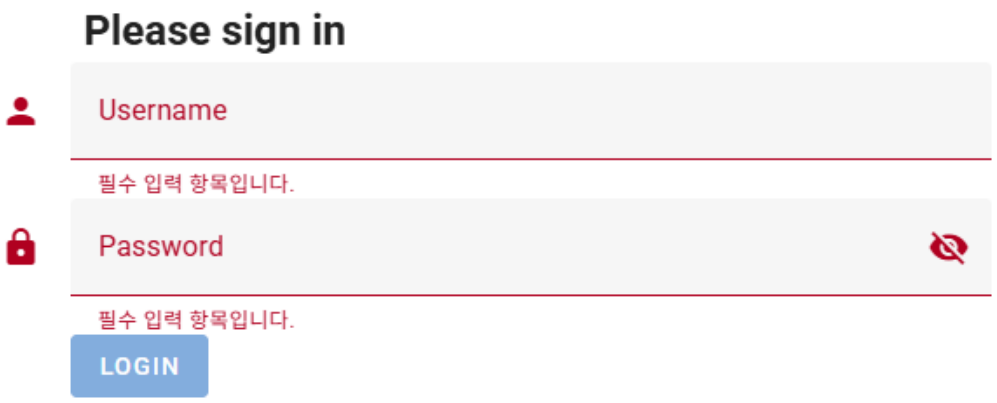
  url에 ws://localhost:9030/stomp/websocket 를 입력하고 Connect를 누릅니다.

  인증서버를 걸어놨기 때문에 아래와 같이 연결 실패가 뜹니다.



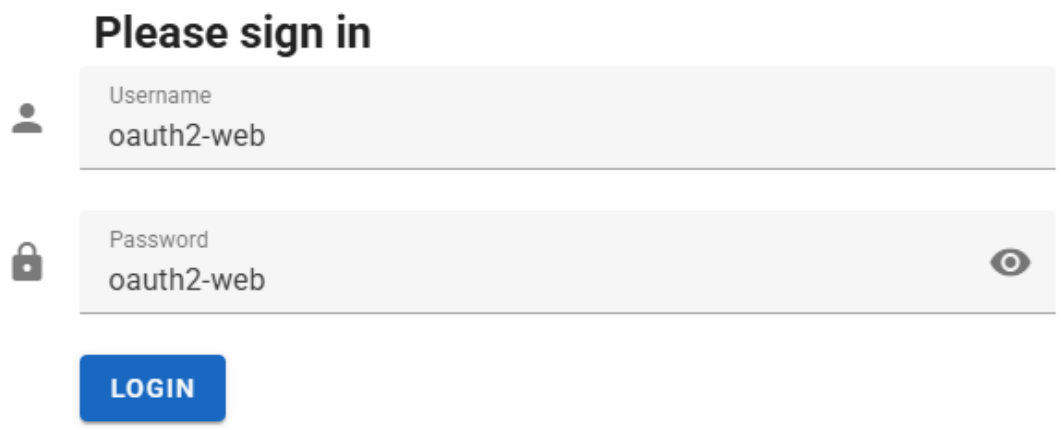
- localhost:9020 | 인증서버로 이동합니다.

  시스템 관리 버튼을 눌러 로그인을 해줍니다.

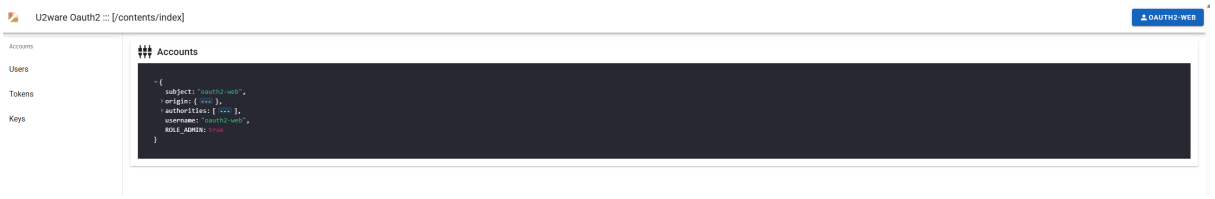


- Username 과 Password는 Dockerfile-run.yml에 oauth2-web 부분에 있습니다.

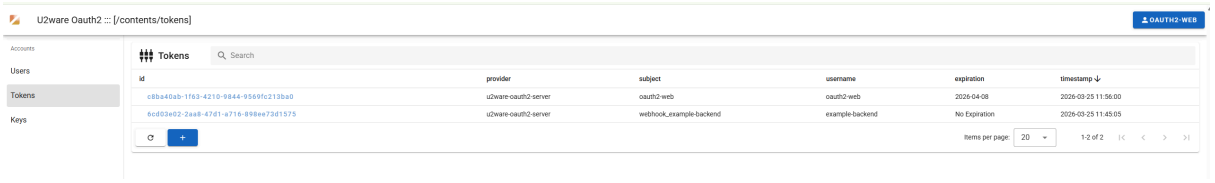
```yaml
##################################################################
# oauth2-web
##################################################################
oauth2-web:
  image: ghcr.io/u2waremanager/io.u2ware.product.oauth2-server:3.5.11

  container_name: oauth2-web

  # volumes:
  #   - <....>:/var/lib/u2ware/logs

  ports:
    - "9020:9020"

  environment:
    server.port: 9020
    spring.security.user.name: oauth2-web
    spring.security.user.password: oauth2-web

    spring.jpa.hibernate.ddl-auto: update
    spring.jpa.properties.hibernate.dialect: org.hibernate.dialect.PostgreSQLDialect

    spring.datasource.driver-class-name: org.postgresql.Driver
```

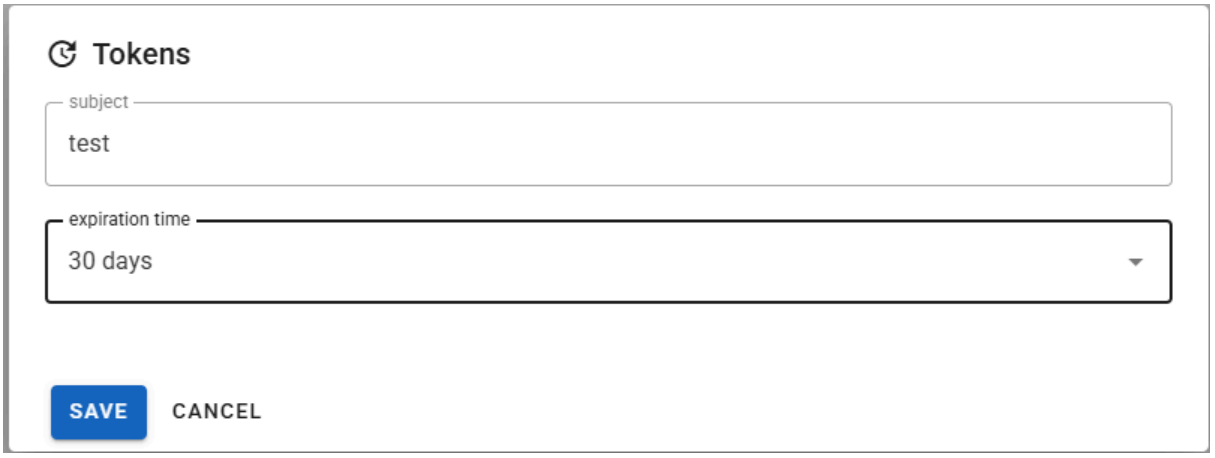


- 로그인 성공 시 아래 사진과 같은 페이지로 이동됩니다.



- Tokens를 눌러 새 token을 생성해줍니다.



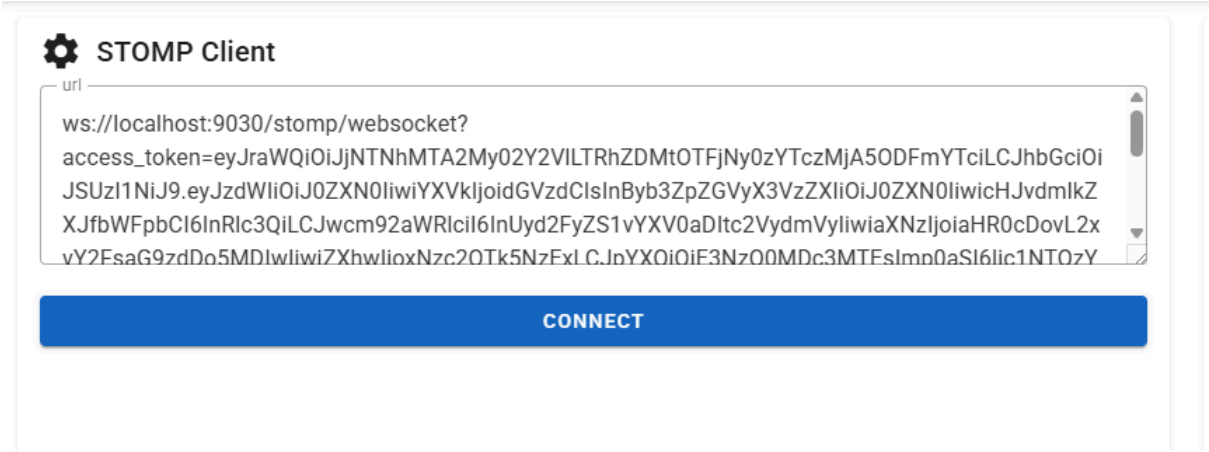


- 만들어진 token의 값을 복사합니다.

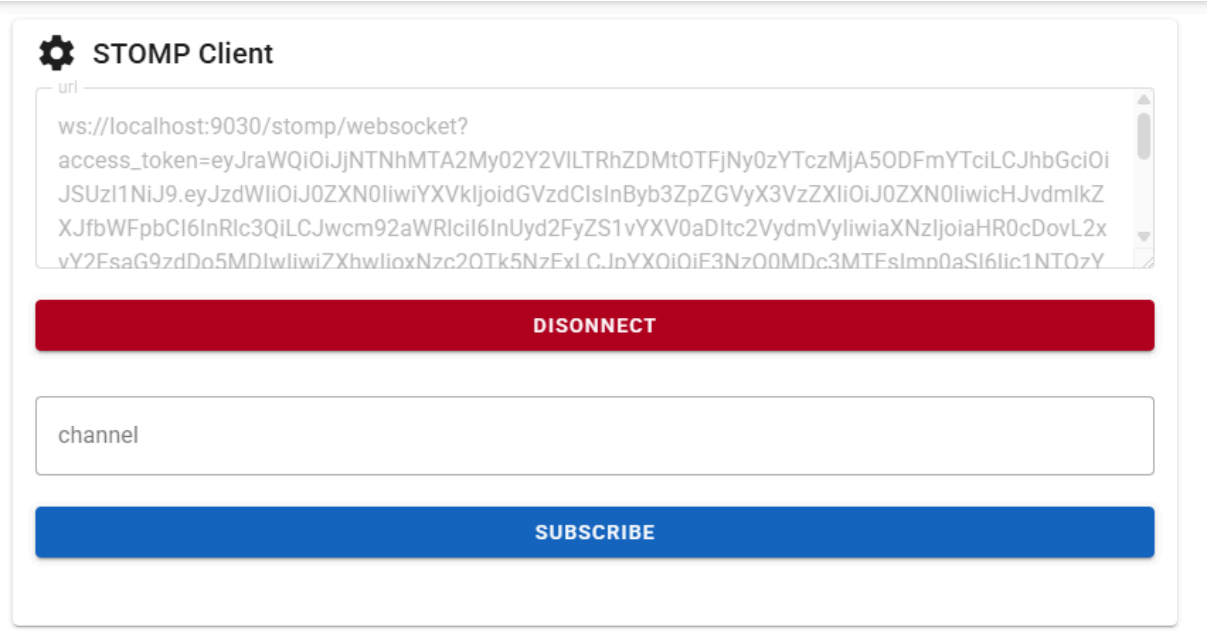
- 다시 localhost:9030으로 이동합니다.
- 아래 사진과 같이 입력해줍니다.

  ws://localhost:9030/stomp/websocket?access_token=

  = 뒤에다가 복사한 토큰 값을 붙여넣어줍니다.



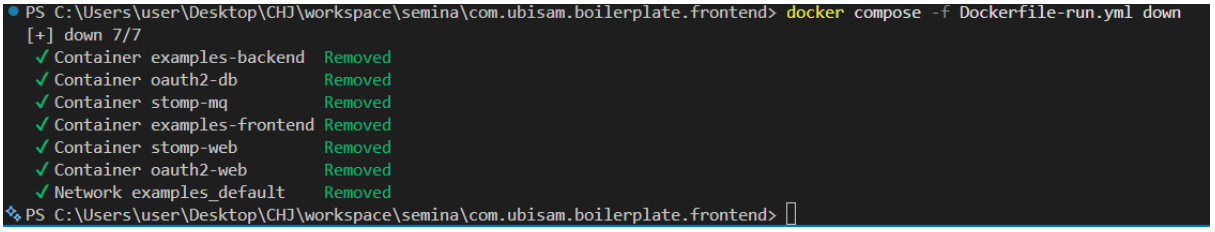
- 그 후 Connect를 누르면 아래 사진과 같이 정상적으로 연결된 것을 확인해 보실 수 있습니다.



- VSCode에서 Frontend 터미널을 열어서 아래 명령어를 입력하여 설치한 Docker를 지웁니다.

💡 docker compose -f Dockerfile-run.yml down

- 아래 사진과 같이 지워지고, Docker Desktop에서도 지워집니다.



Google 로그인 추가

- 로그인 옵션에 Google 로그인을 추가해보겠습니다.

  Dockerfile-run.yml의 oauth2-web 부분으로 이동합니다.

```yaml
environment:
  server.port: 9020
  spring.security.user.name: oauth2-web
  spring.security.user.password: oauth2-web

  spring.jpa.hibernate.ddl-auto: update
  spring.jpa.properties.hibernate.dialect: org.hibernate.dialect.PostgreSQLDialect

  spring.datasource.driver-class-name: org.postgresql.Driver
  spring.datasource.url: jdbc:postgresql://oauth2-db:5432/oauth2-db
  spring.datasource.username: oauth2-db
  spring.datasource.password: oauth2-db

  io.u2ware.common.oauth2.clients.example-backend: http://examples-backend:8080/oauth2/token
```

아래 environment: 부분에 2줄을 추가합니다.

> 💡 spring.security.oauth2.client.registration.google.client-id:
> spring.security.oauth2.client.registration.google.client-secret:

- id랑 secret에는 이전 세미나에서 받았던 google console 쪽 id와 secret을 넣어줍니다.

```yaml
environment:
  server.port: 9020
  spring.security.user.name: oauth2-web
  spring.security.user.password: oauth2-web

  spring.jpa.hibernate.ddl-auto: update
  spring.jpa.properties.hibernate.dialect: org.hibernate.dialect.PostgreSQLDialect

  spring.datasource.driver-class-name: org.postgresql.Driver
  spring.datasource.url: jdbc:postgresql://oauth2-db:5432/oauth2-db
  spring.datasource.username: oauth2-db
  spring.datasource.password: oauth2-db

  spring.security.oauth2.client.registration.google.client-id: 530528304495-bddbugj6r4114m45lv3imojdod6ool4k.apps.googleusercontent.com
  spring.security.oauth2.client.registration.google.client-secret: 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

  io.u2ware.common.oauth2.clients.example-backend: http://examples-backend:8080/oauth2/token
```

- 그 후 다시 Frontend 터미널에서 docker compose -f Dockerfile-run.yml up -d 를 실행해주어

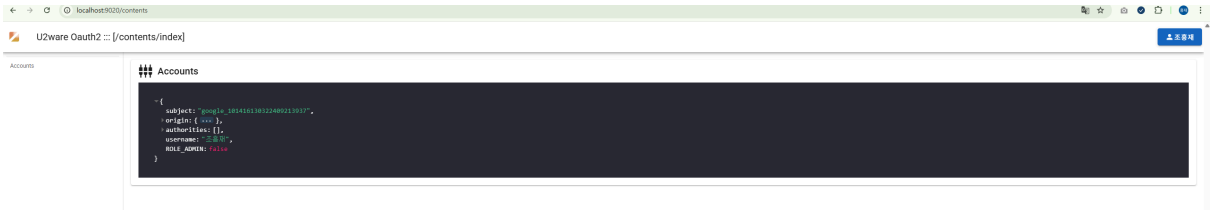
  Docker Container를 열어줍니다.

- localhost:9020 으로 이동해서 시스템관리 버튼을 클릭 시 아래 사진과 같이 옵션이 추가됩니다.

- 구글 로그인을 해줍니다.

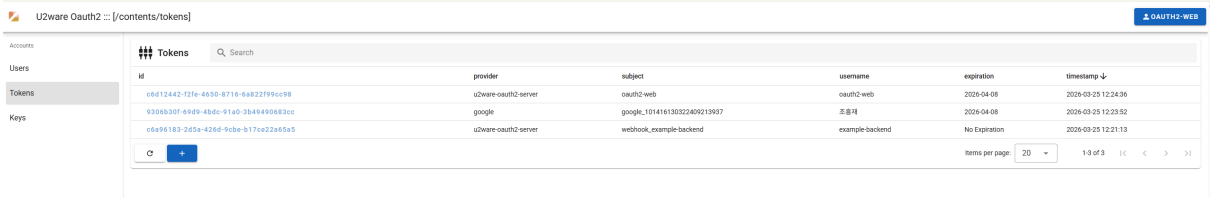
  성공적으로 로그인 시 아래 사진과 같이 로그인이 됩니다.

  구글 로그인은 ADMIN이 아니라 USER 권한 입니다.

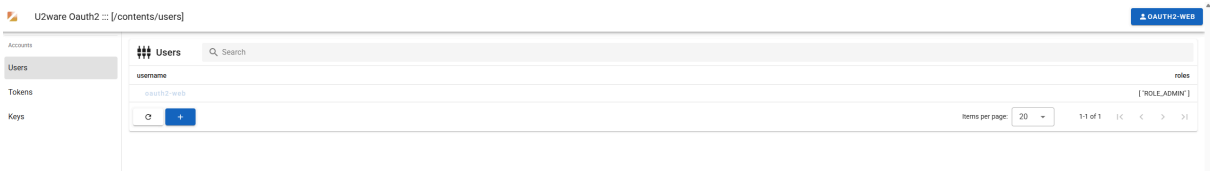


- 구글 로그아웃 후 oauth2-web으로 다시 로그인합니다.

  Tokens 탭을 눌러 이동하면 google 로 로그인한 사용자가 보입니다.



- User 를 추가해보겠습니다. Users 탭을 눌러 이동합니다.



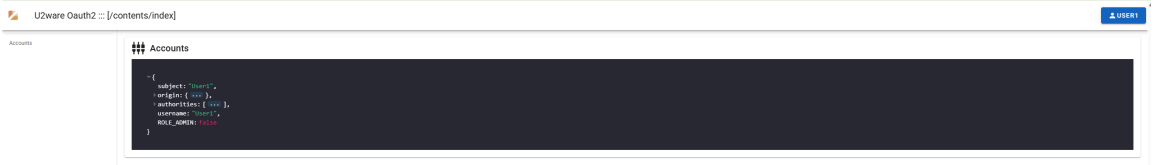
- User1, User2 를 추가합니다.

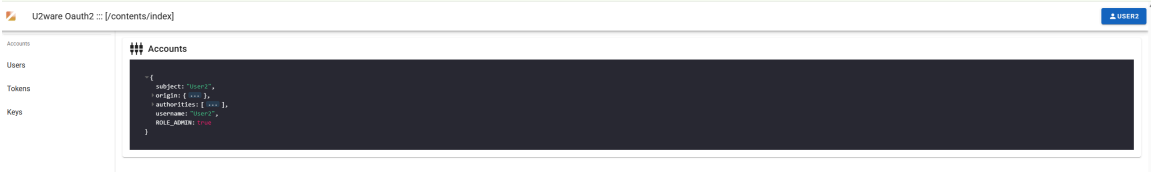
  User1은 ROLE_USER 권한을 부여한 후 추가합니다.

User2는 ROLE_ADMIN 권한을 부여한 후 추가합니다.

- 로그아웃 후 각각 로그인하여 User권한과 Admin 권한의 차이점을 비교해봅니다.

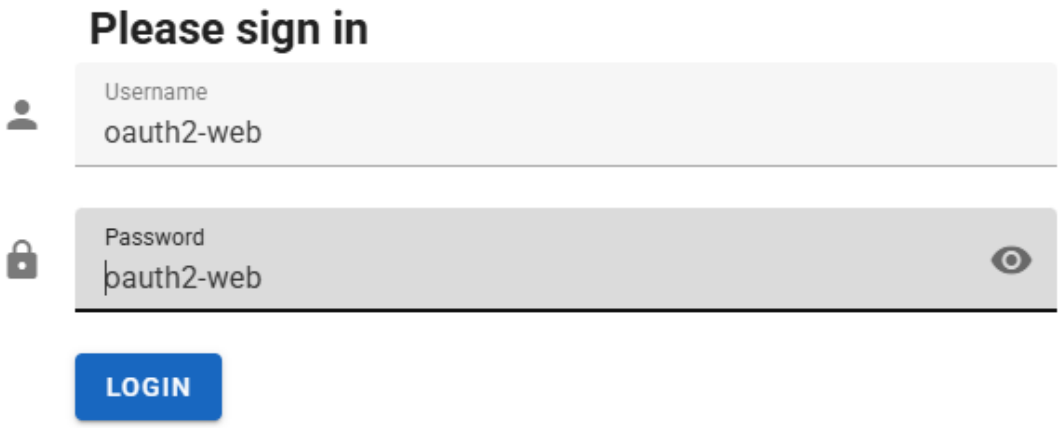
  아래 사진과 같이 User1은 왼쪽 메뉴가 없는 반면, User2는 있는 것을 확인해 보실 수 있습니다.



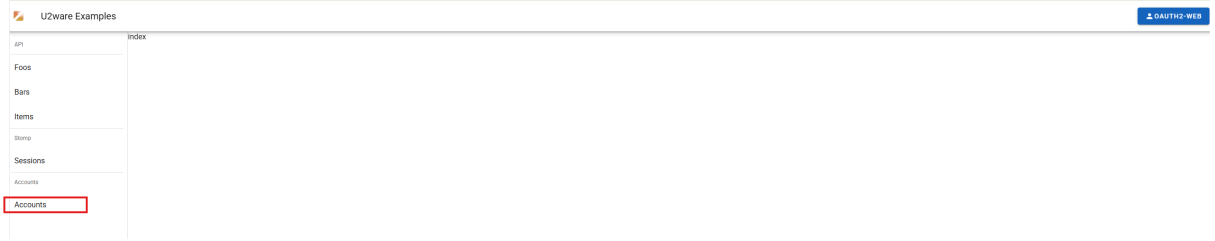


- 다음으로 localhost:3000 으로 이동합니다.

  아래 사진대로 로그인을 해줍니다.

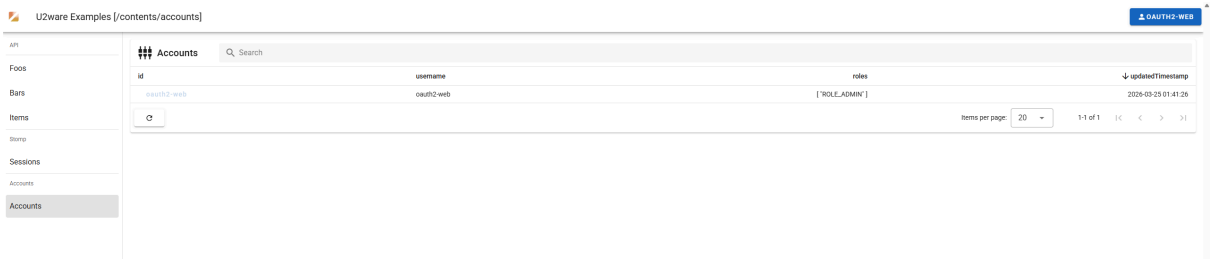


- 그 후 왼쪽 Accounts 메뉴를 선택합니다.



- 아래 사진처럼 username: oauth2-web이 updatedTimestamp 시간부터

  oauth2를 거쳐서 로그인 후 frontend를 사용하기 시작했습니다 라는 의미를 가진 로그입니다.

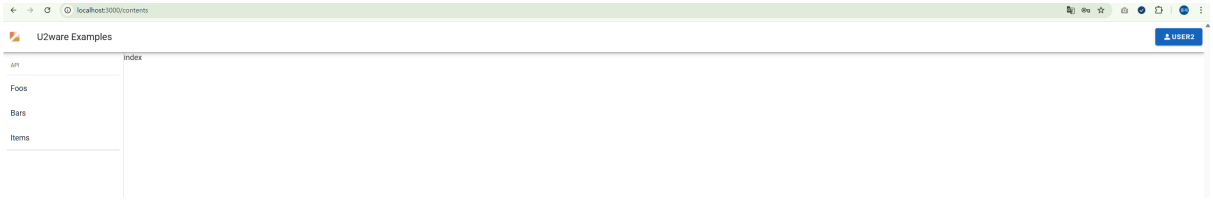


- 로그아웃 후 User1으로 로그인 합니다

  권한이 User이기 때문에 옆에 메뉴가 일부 보이지 않습니다.

- 이번엔 User2로 로그인합니다.

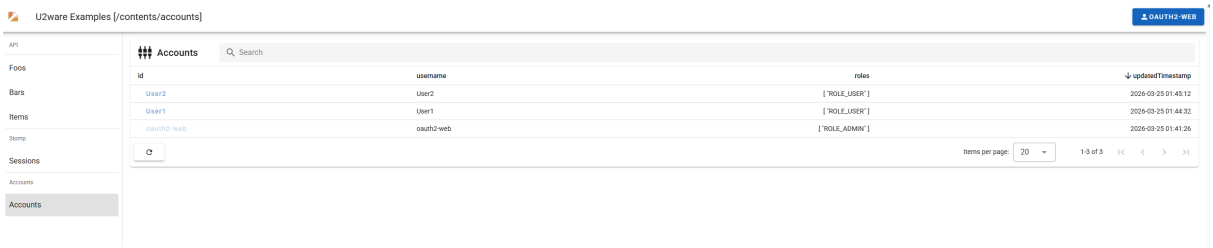
  아래 사진과 같이 admin권한을 위에서 부여하였는데도 User1과 같다는 것을 확인해 보실 수 있습니다.



- 다시 oauth2-web으로 로그인합니다.

  그 후 Accounts 메뉴를 선택하면 아래 사진과 같이 User1, User2가 권한이 ROLE_USER인

  것을 확인해 보실 수 있습니다.



🔐 개념 정리

- **oauth2-web에서 부여한 admin 권한**은
  👉 *인증 서버(Authentication Server)에 대한 권한*입니다.
- 하지만 **프론트(클라이언트 서버)**는
  👉 별도의 리소스 서버이기 때문에
  👉 **해당 권한이 자동으로 적용되지 않습니다.**

즉,

> 인증 서버에서 admin이라고 해서
>
> 클라이언트 서버에서도 admin이 되는 것은 아닙니다.

📌 비유 (네이버 예시)

- 저는 하나의 **네이버 계정**을 가지고 있습니다.
- 이 계정에는 서로 다른 서비스에 대한 권한이 있습니다:
  - **블로그 → 관리자(admin) 권한**
  - **카페 → 일반 사용자(user) 권한**

👉 결과

- 블로그에서는
  → 글 작성 / 수정 / 삭제 **모두 가능 (admin 권한)**
- 카페에서는
  → 일부 제한된 기능만 가능 (user 권한)

실제 Code

Backend

- VSCode를 다시 켜서 저희 코드를 알아가보겠습니다.

- 먼저 domain 폴더를 보시면

  새로 auditing, exception, properties, security가 추가된 것을 확인해 보실 수 있습니다.

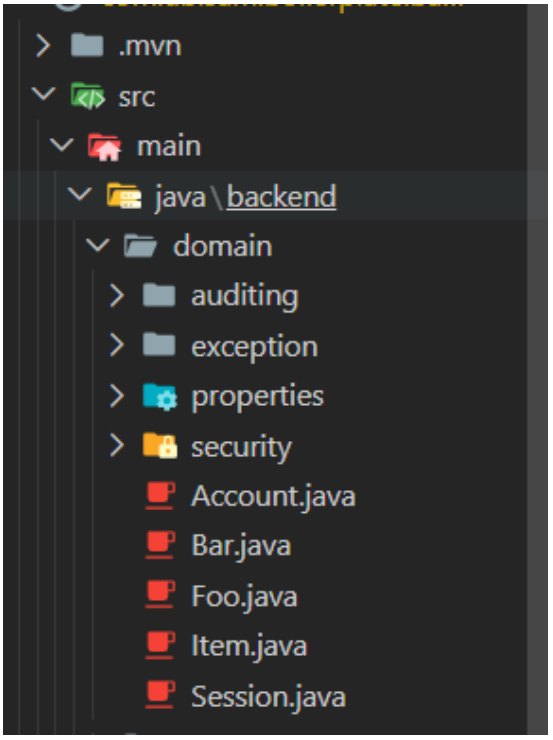
  해당 기능은 프로젝트 생성하실 때 고대로 복사해서 사용하시면 됩니다.

- 다음으로 아래 .java에 대한 내용입니다

  보는 순서는 Foo → Bar → Item 순서입니다.

- Foo와 Bar의 형태는 저희에게 익숙한

  Hello, World라고 생각하시면됩니다.



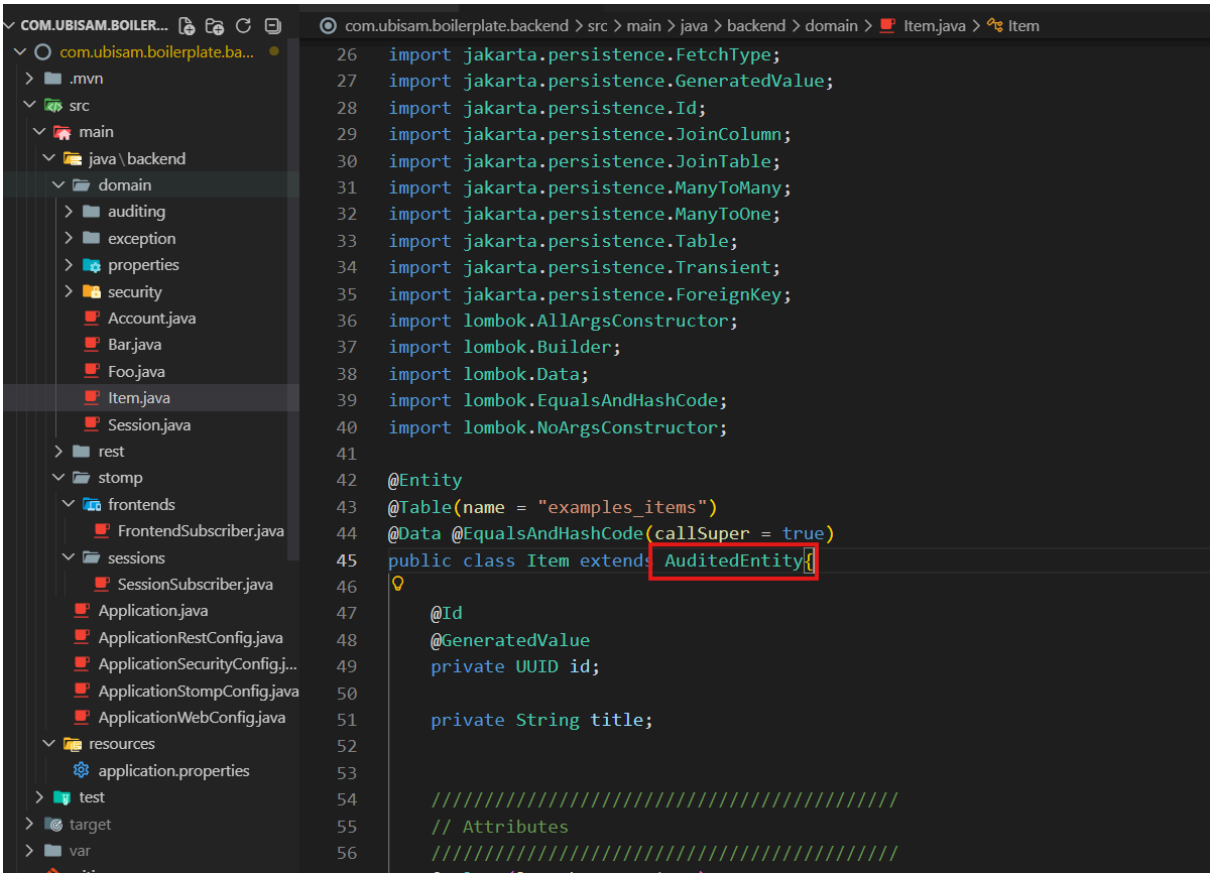
- Item.java로 가보겠습니다.

  Item은 특이하게 AuditedEntity를 상속받고 있습니다.

  AuditedEntity는 DB에 누가 DB를 업데이트했고, IP는 무엇이고 등 정보를 자동으로 넣어주는 클래스입니다.

  - 생성 시간 (`createdAt`)
  - 생성자 (`createdBy`)
  - 수정 시간 (`updatedAt`)
  - 수정자 (`updatedBy`)

  이걸 매 엔티티마다 반복해서 작성하면 비효율적이기 때문에 **공통 클래스로 빼서 상속**하는 구조를 사용합니다.



- 해당 클래스를 Ctrl + 좌클릭해서 이동해보면 아래 사진과 같은 코드가 나오는 것을 확인해 보실 수 있습니다.

```
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public @Data abstract class AuditedEntity {

    @CreatedBy
    @Embedded
    @AttributeOverrides({
            @AttributeOverride(name = "username", column = @Column(name = "inserted_username")),
            @AttributeOverride(name = "address", column = @Column(name = "inserted_address")),
            @AttributeOverride(name = "timestamp", column = @Column(name = "inserted_timestamp"))
    })
    @JsonProperty(access = JsonProperty.Access.READ_ONLY)
    protected Audited inserted;

    @LastModifiedBy
    @Embedded
    @AttributeOverrides({
            @AttributeOverride(name = "username", column = @Column(name = "updated_username")),
            @AttributeOverride(name = "address", column = @Column(name = "updated_address")),
            @AttributeOverride(name = "timestamp", column = @Column(name = "updated_timestamp"))
    })
    @JsonProperty(access = JsonProperty.Access.READ_ONLY)
    protected Audited updated;

}
```

- 설정한 6개의 필드가 자동으로 생성되고 입력됩니다.

- 다음으로 Cryptovalue 입니다.
- `cryptoValue`는 암호화가 필요한 데이터를 안전하게 저장·관리하기 위한 필드입니다.

```java
@Entity
@Table(name = "examples_items")
@Data @EqualsAndHashCode(callSuper = true)
public class Item extends AuditedEntity{

    @Id
    @GeneratedValue
    private UUID id;

    private String title;


    ////////////////////////////////////////////
    // Attributes
    ////////////////////////////////////////////
    @Column(length = 1024*100)
    @Convert(converter=CryptoConverter.class)
    private String cryptoValue;

    @Column(length = 1024*100)
    private AttributesMap jsonValue;


    @Column(length = 1024*100)
    private AttributesSet arrayValue;
```

- 다음은 AttributesMap 입니다.

  `AttributesMap`은 엔티티에서 JSON 형태의 동적 속성, 확장 필드, 사용자 정의 데이터 등을 저장할 때 활용할 수 있는 범용적인 맵 타입입니다. key-value 형태로 넣을 수 있습니다.

```java
@Entity
@Table(name = "examples_items")
@Data @EqualsAndHashCode(callSuper = true)
public class Item extends AuditedEntity{

    @Id
    @GeneratedValue
    private UUID id;

    private String title;


    ////////////////////////////////////////////
    // Attributes
    ////////////////////////////////////////////
    @Column(length = 1024*100)
    @Convert(converter=CryptoConverter.class)
    private String cryptoValue;

    @Column(length = 1024*100)
    private AttributesMap jsonValue;


    @Column(length = 1024*100)
    private AttributesSet arrayValue;
```

- 다음으로 AttributesSet입니다.

  `AttributesSet`은 엔티티에서 JSON 배열 형태의 동적 속성, 확장 필드, 사용자 정의 데이터 등을 저장할 때 활용할 수 있는 범용적인 Set 타입입니다. 배열 형태로 넣을 수 있습니다.

```java
@Entity
@Table(name = "examples_items")
@Data @EqualsAndHashCode(callSuper = true)
public class Item extends AuditedEntity{

    @Id
    @GeneratedValue
    private UUID id;

    private String title;


    ////////////////////////////////////////////
    // Attributes
    ////////////////////////////////////////////
    @Column(length = 1024*100)
    @Convert(converter=CryptoConverter.class)
    private String cryptoValue;

    @Column(length = 1024*100)
    private AttributesMap jsonValue;


    @Column(length = 1024*100)
    private AttributesSet arrayValue;
```

- 다음은 ManyToOne 관계인 Foo입니다.

  Item은 하나의 Foo를 가집니다.

  `@JsonProperty(access = Access.READ_ONLY)`: API 응답에서만 읽을 수 있고, 클라이언트가 직접 값을 변경할 수 없습니다. 즉, 조회 전용입니다.

  `@Transient @JsonProperty(access = Access.WRITE_ONLY):` 이 필드는 DB에 저장되지 않으며, API 요청(쓰기)에서만 사용됩니다. 클라이언트가 `foo` 와의 관계를 링크(URL)로 지정할 때 활용됩니다.

```
//////////////////////////////////////////////
// ManyToOne
// {foo : "http://....."} or {foo : {id : "", name: "", _links : {}}}"
//////////////////////////////////////////////
@ManyToOne
@JoinColumn(foreignKey=@ForeignKey(ConstraintMode.NO_CONSTRAINT))
@RestResource(exported = false)
@JsonProperty(access = Access.READ_ONLY)
private Foo foo;

@Transient
@JsonProperty(access = Access.WRITE_ONLY)
private Link fooLink;
```

- 다음은 ManyToMany 관계인 Bar입니다.

  Item은 여러개의 Bar를 가집니다.

  `@JoinTable(foreignKey=@ForeignKey(ConstraintMode.NO_CONSTRAINT), inverseForeignKey=@ForeignKey(ConstraintMode.NO_CONSTRAINT))` :

  중간 테이블을 생성하지만, 외래키 제약조건은 DB에 생성하지 않습니다.

  `private Set<Bar> bars = new HashSet<>();` : 실제 연관된 `Bar` 객체들의 집합(읽기 전용, DB 매핑).

  `@Transient @JsonProperty(access = Access.WRITE_ONLY) private Set<Link> barsLinks = new HashSet<>();` : DB에는 저장되지 않으며, API 요청(쓰기)에서만 사용됩니다. 클라이언트가 여러 `Bar` 와의 관계를 링크(URL)로 지정할 때 활용됩니다.

```
//////////////////////////////////////////////
// ManyToMany
// {bar : "http://....."} or {bar : {id : "", name: "", _links : {}}}"
//////////////////////////////////////////////
@ManyToMany(fetch=FetchType.EAGER)
@JoinTable(foreignKey=@ForeignKey(ConstraintMode.NO_CONSTRAINT), inverseForeignKey=@ForeignKey(ConstraintMode.NO_CONSTRAINT))
@RestResource(exported = false)
private Set<Bar> bars = new HashSet<>();

@Transient
@JsonProperty(access = Access.WRITE_ONLY)
private Set<Link> barsLinks = new HashSet<>();
```

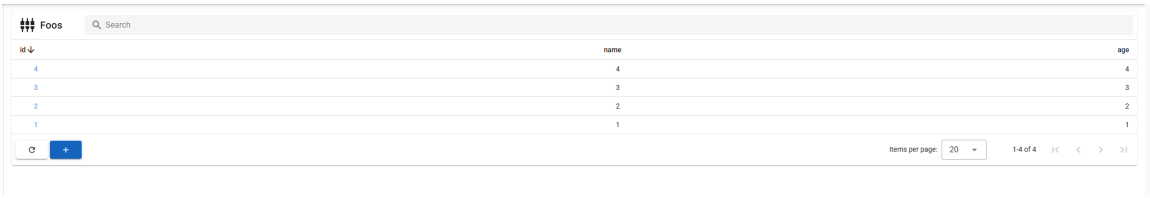
- 다음은 ElementCollection 입니다.

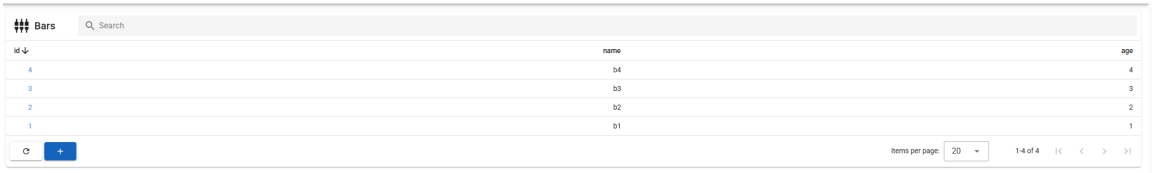
  Item은 여러개의 Child를 가집니다.

  `@ElementCollection` : 엔티티가 별도의 엔티티가 아닌 값 타입(여기서는 `Child`)의 컬렉션을 가질 수 있게 합니다. 즉, `Child` 는 독립적인 엔티티가 아니라 `Item` 에 종속된 값 객체입니다.

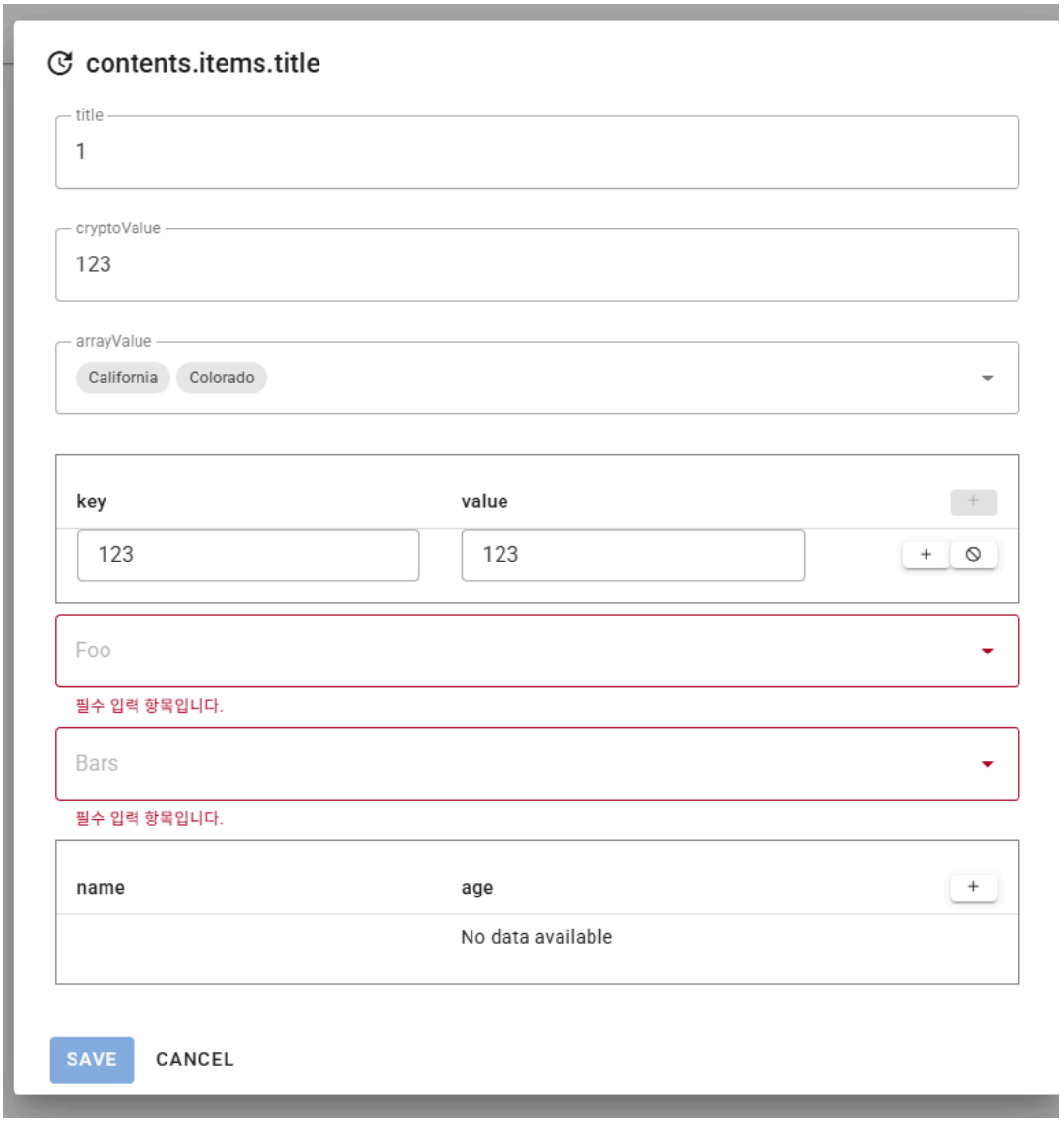
- @ElementCollection은 부모 엔티티와 생명주기(LifeCycle)가 동일한 값 타입 컬렉션(예: 메일의 첨부파일처럼 함께 저장·삭제되는 종속 데이터)에 사용합니다.

- 직접 테스트를 위해 localhost:3000 으로 이동하여 oauth2-web 으로 로그인해줍니다.
  - Foo와 Bar에 더미 데이터를 넣어줍니다.



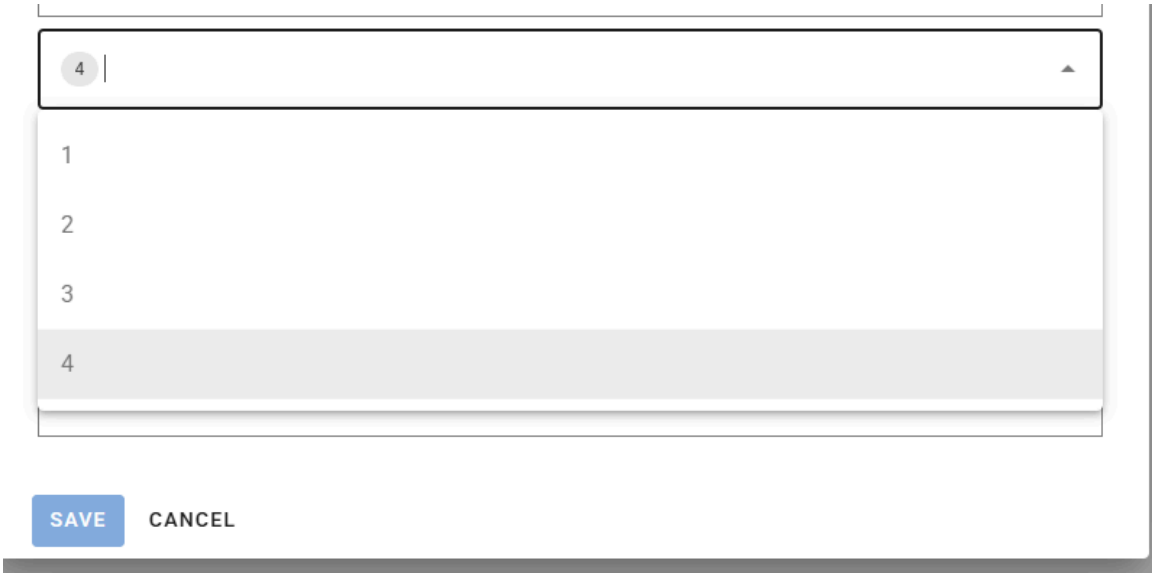


- Item 메뉴로 넘어와서 Item을 추가해줍니다.
  - title, crptoValue, arrayValue, AttributesMap은 원하는대로 넣어줍니다.

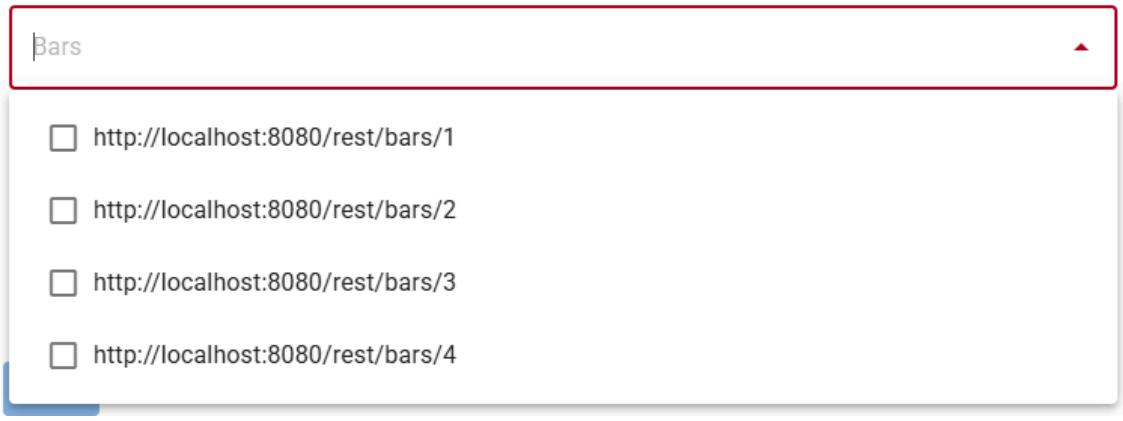


- Foo는 아래 사진을 기준으로 4개가 있습니다.

  하지만 ManyToOne 이기때문에 1개만 선택됩니다.



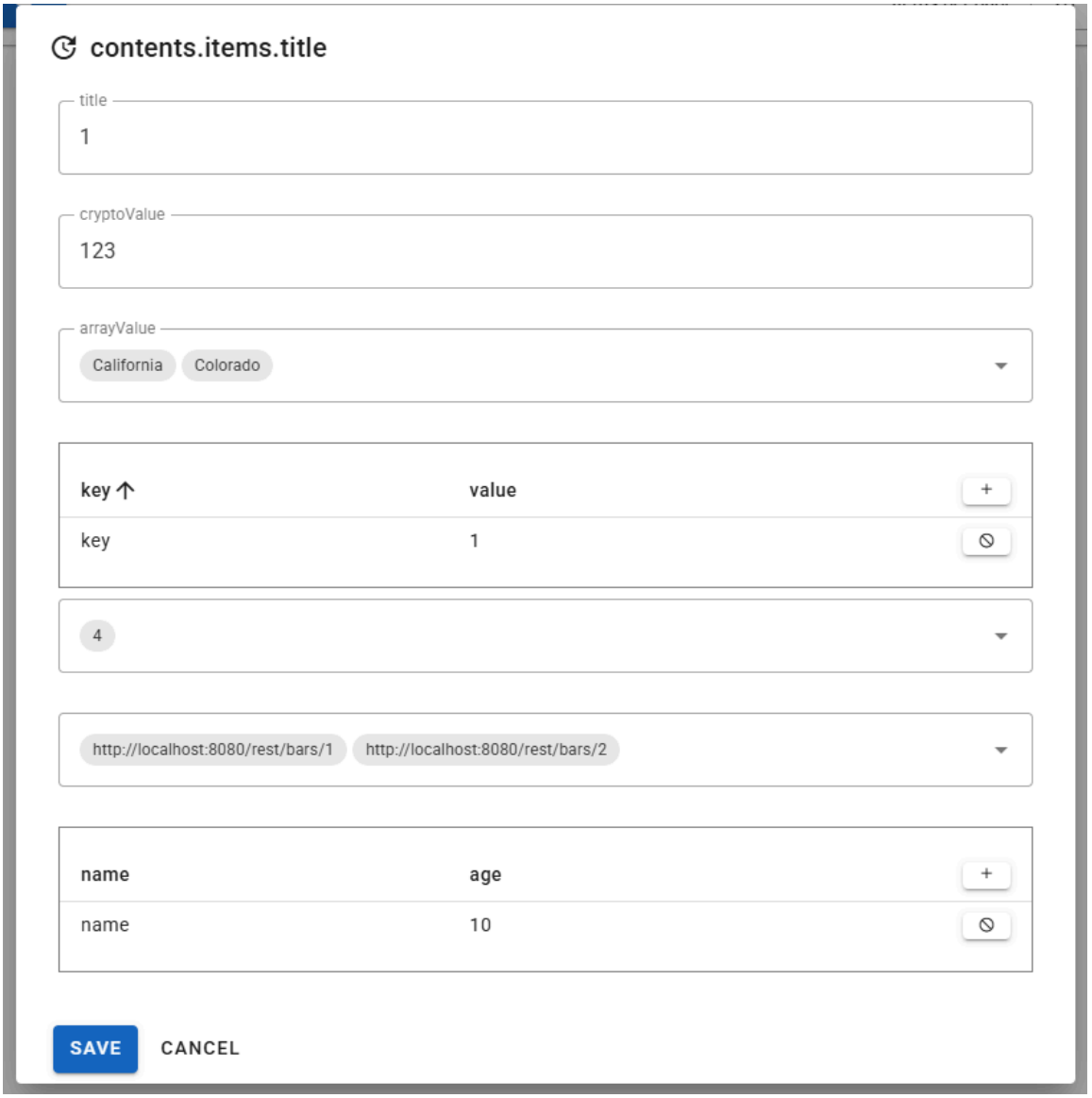
- Bars는 ManyToMany 관계이기 때문에 여러개 선택이 가능합니다.



- 그 아래는 ElementCollection 입니다.

  원하는 값을 추가해줍니다. age는 숫자만 가능합니다.

- 최종적으로 아래 사진과 같이 설정되었고 save를 눌러 item을 추가합니다.



- 아래 사진과 같이 item이 1개 추가된 것을 확인해 보실 수 있습니다.

- 해당 item을 삭제할 시 Foo, Bar를 제외한 나머지는 싹 삭제가 됩니다.

  아래 사진과 같이 DELETE 버튼을 눌러봅니다.

contents.items.title

id
1b03076b-74bd-4368-86a5-7de75a1fc3e7

title
1

cryptoValue
필수 입력 항목입니다.

arrayValue
Colorado California

key	value
key	1

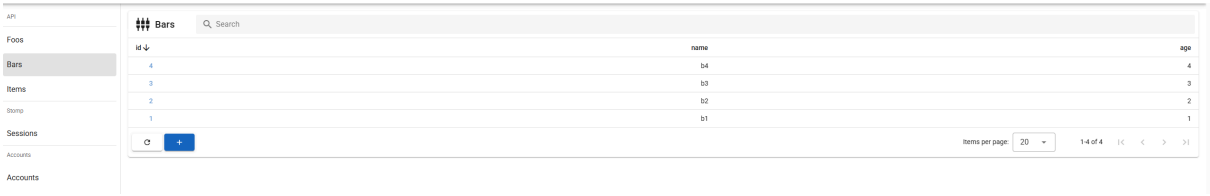
4

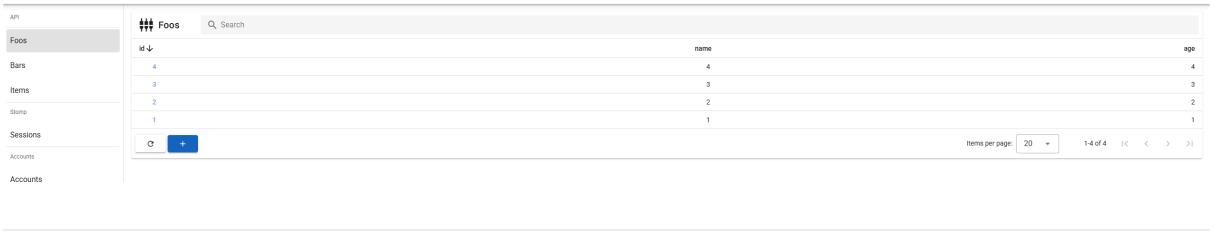
http://localhost:8080/rest/bars/1 http://localhost:8080/rest/bars/2

name	age
name	10

SAVE CANCEL DELETE

- 그 후 Foo, Bar 메뉴를 선택해 확인하면 그대로 남아있는 모습을 확인해 보실 수 있습니다.

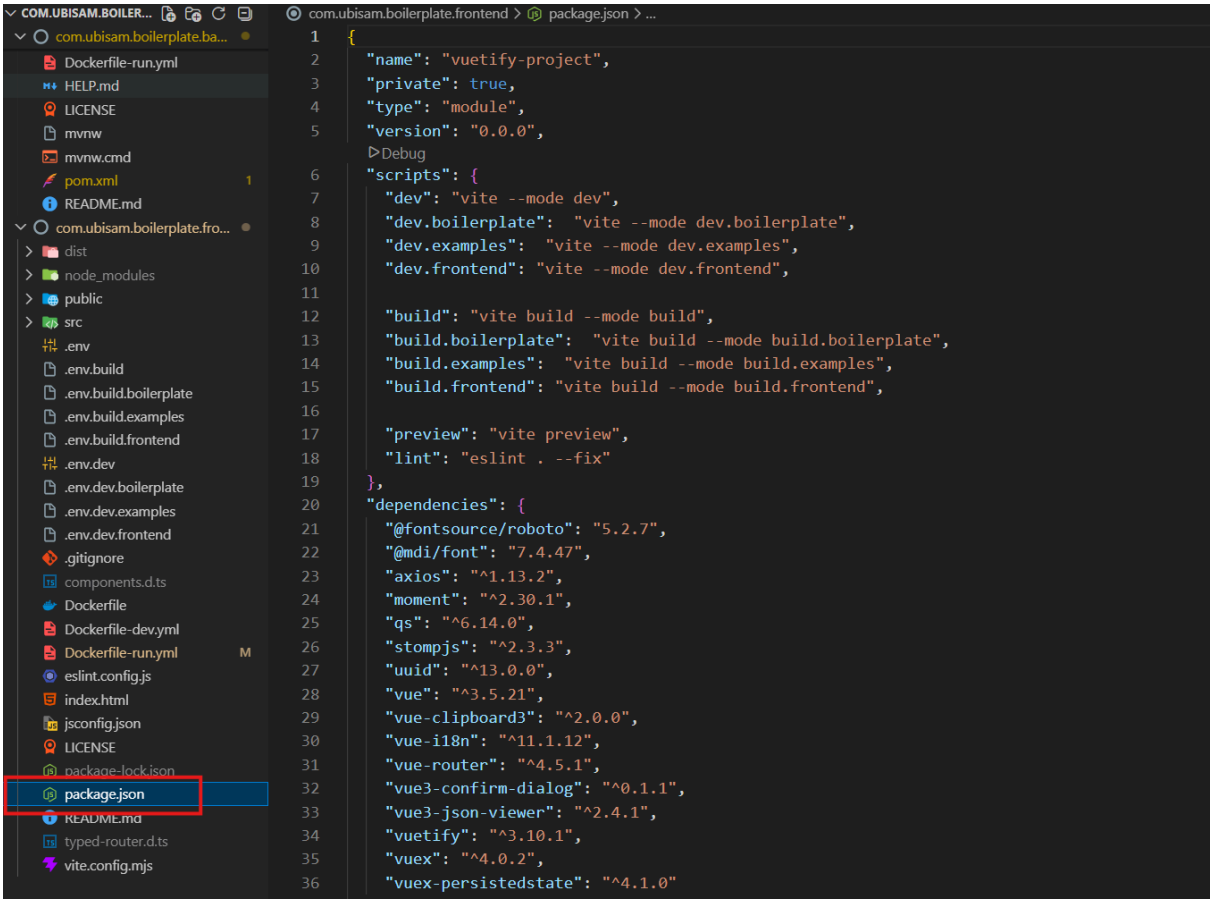




Frontend

- 해당 내용을 Frontend에서 어떤 방식으로 구현했는지를 확인해 보겠습니다.

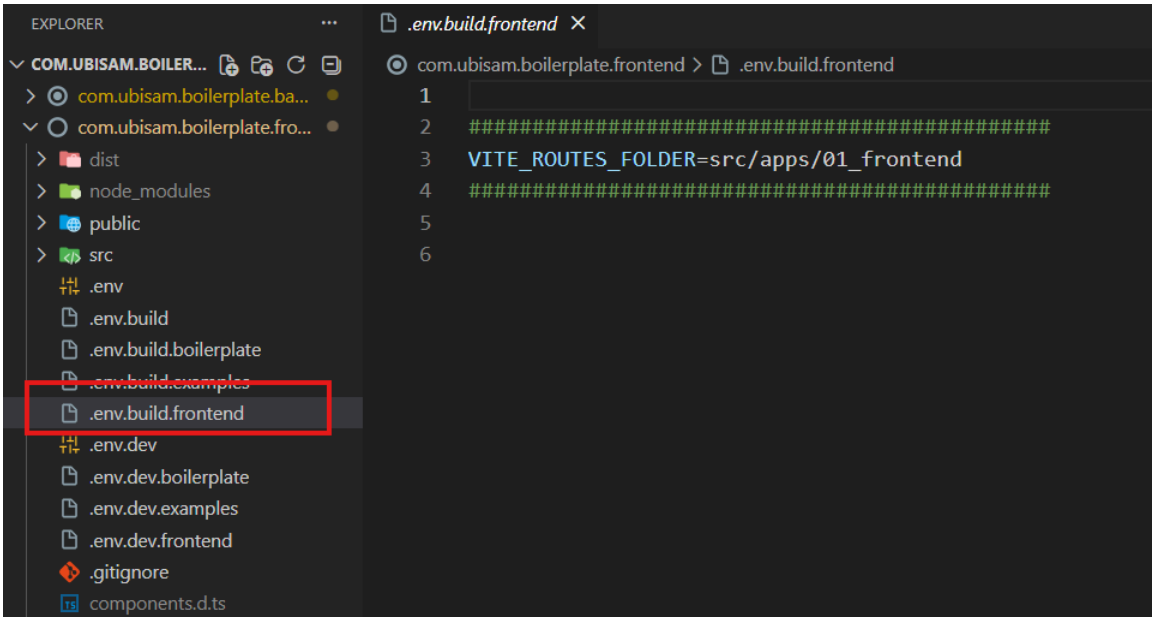
  VSCode를 열어서 Frontend의 package.json을 열어줍니다.



- build.frontend 를 확인합니다.

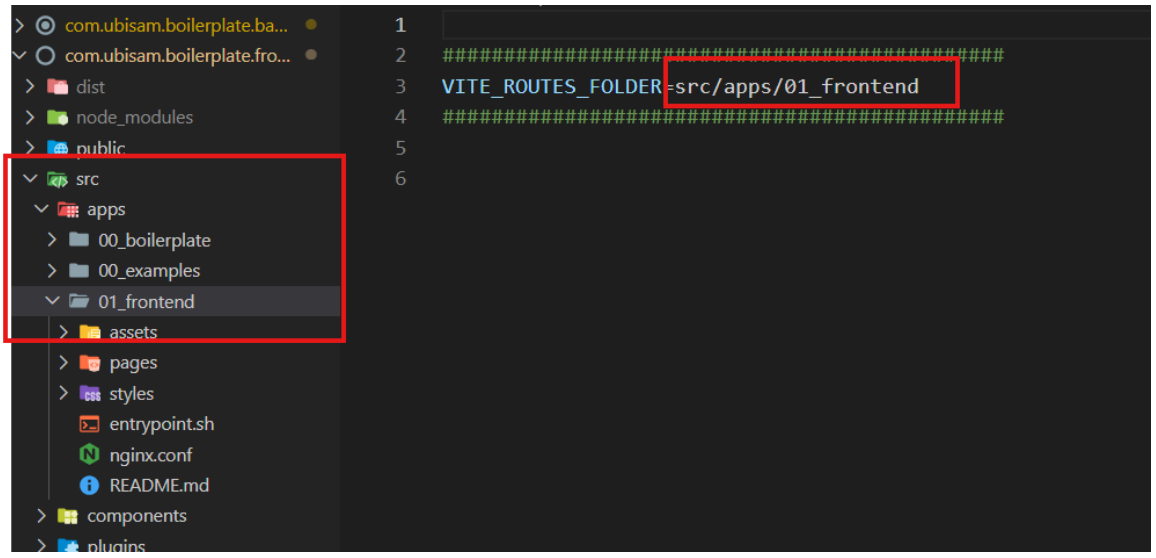
```
"build": "vite build --mode build",
"build.boilerplate": "vite build --mode build.boilerplate",
"build.examples": "vite build --mode build.examples",
"build.frontend": "vite build --mode build.frontend",
```

- .env.build.frontend를 열어줍니다.



- 프로젝트의 root 주소가 나옵니다.

  해당 폴더를 열어줍니다.



- 해당 방식으로 설계한 이유는 plugins와 components를 같이 쓰기 위해서입니다.

```
VITE_ROUTES_FOLDER=src/apps/01_frontend
##############################################
```

- 아까 저희가 추가한 items는 src/apps/01_frontend/pages/contents/items.vue 페이지입니다.

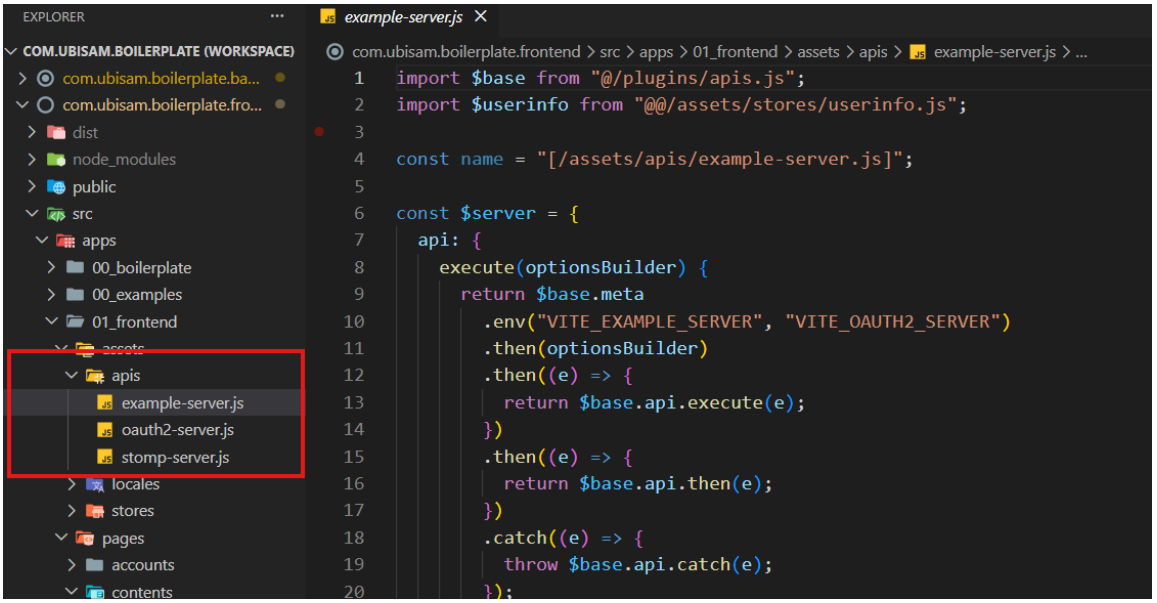
```
com.ubisam.boilerplate.frontend > src > apps > 01_frontend > pages > contents > items.vue > { } template
<template>
  <v-container class="pa-4" fluid>
    <v-card>
      <v-data-table-server
        </template>
        <!--
        ////////////////////////////
        # Table Cell Template End
        ////////////////////////////
        -->

        <template v-slot:footer.prepend>
          <v-btn class="ms-1" text variant="elevated" @click="refreshAction
            <v-icon>mdi-refresh</v-icon>
          </v-btn>

          <v-btn
            class="ms-1"
            text
            variant="elevated"
            color="primary"
            @click="newAction"
          >
            <v-icon>mdi-plus</v-icon>
          </v-btn>

          <v-spacer></v-spacer>
        </template>
      </v-data-table-server>

      <v-dialog v-model="dialog" persistent width="800">
        <v-card
          prepend-icon="mdi-update"
          :title="$t('contents.items.title')"
          :subtitle="editForm.name"
        >
          <v-card-text>
```

- 다음으로 assets 폴더입니다.

  assets 폴더를 열어보시면 3개의 .js 파일이 있는데

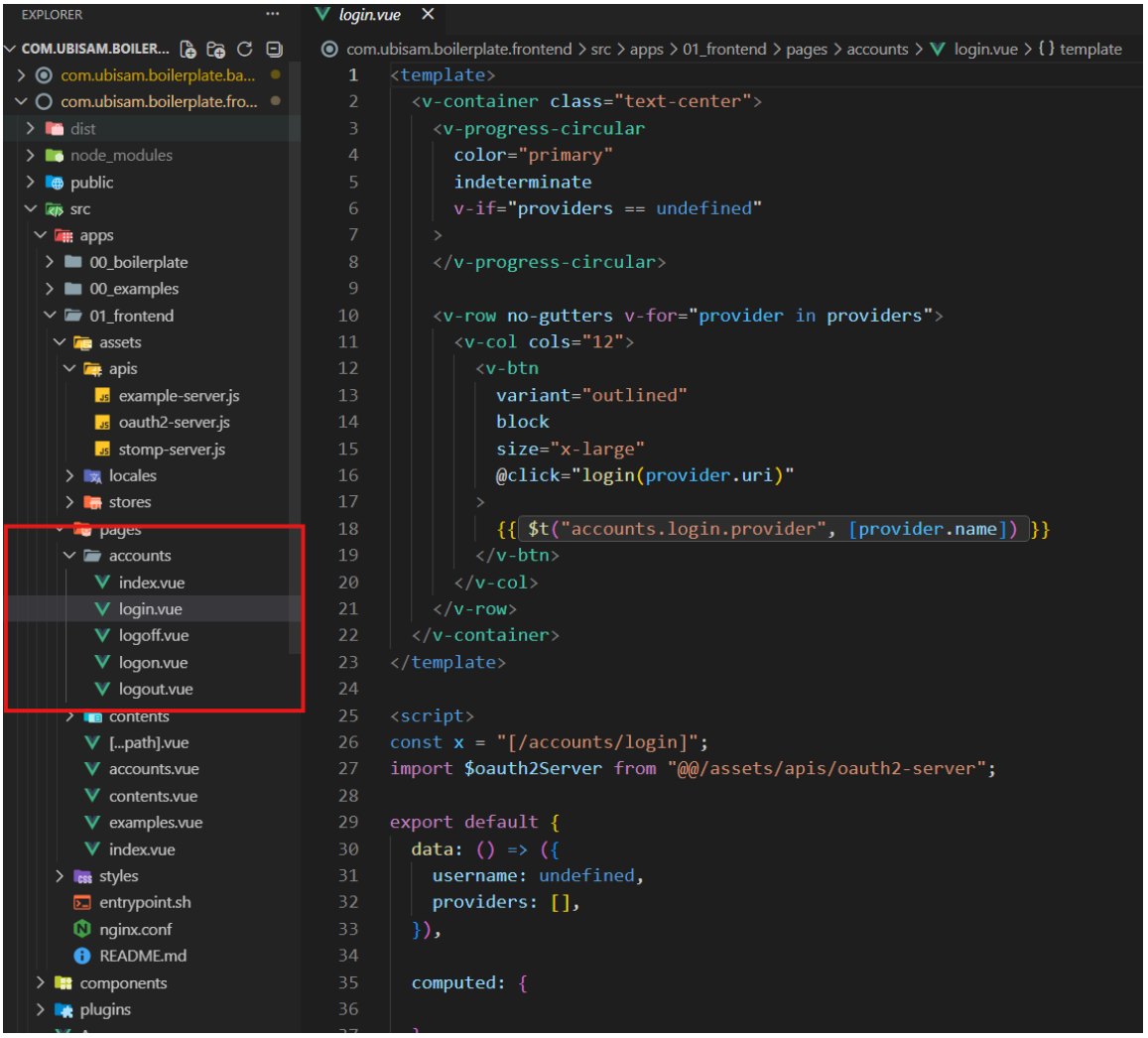
  저희가 연결해야할 서버를 예쁘게 선언해놓은 것입니다.

  즉, 하드코딩 하지말고 백엔드 서버별로 모아서 정리를 하는 것입니다.



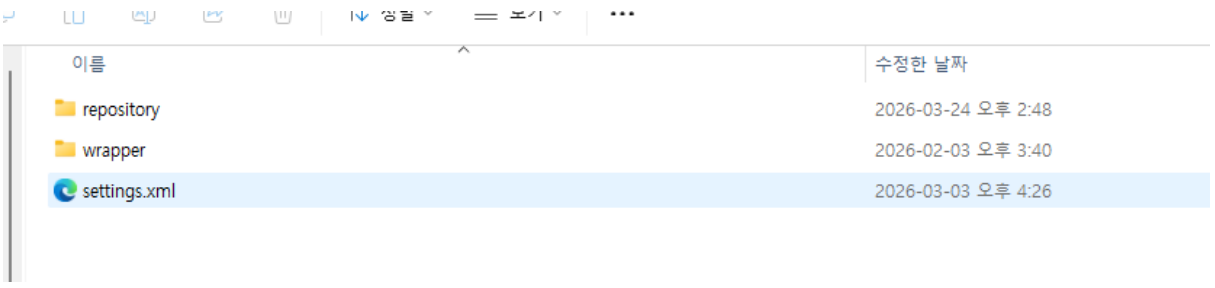
- 다음은 accounts 폴더입니다.

  해당 폴더는 Oauth2 클라이언트 입니다. 즉, Oauth2-web이 됩니다.



BoilerPlate 관련 오류 해결

- VSCode를 완전히 종료합니다.
- C:\Users\user\.m2 로 이동합니다. settings.xml 파일을 아래 내용으로 변경합니다.




```
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
  xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.
0
                      http://maven.apache.org/xsd/settings-
1.0.0.xsd">

  <activeProfiles>
    <activeProfile>github</activeProfile>
  </activeProfiles>

  <profiles>
    <profile>
      <id>github</id>
      <repositories>
        <repository>
          <id>central</id>
          <url>https://repo.maven.apache.org/maven2</url>
        </repository>

        <repository>
          <id>github</id>
          <url>https://maven.pkg.github.com/u2waremanager/i
o.u2ware.common.u2ware-common-data</url>
          <snapshots>
            <enabled>true</enabled>
          </snapshots>
        </repository>

        <repository>
          <id>github</id>
          <url>https://maven.pkg.github.com/u2waremanager/i
o.u2ware.common.u2ware-common-stomp</url>
          <snapshots>
            <enabled>true</enabled>
          </snapshots>
        </repository>

        <repository>
          <id>github</id>
```

```
          <url>https://maven.pkg.github.com/u2waremanager/i
o.u2ware.common.u2ware-common-oauth2</url>
          <snapshots>
            <enabled>true</enabled>
          </snapshots>
        </repository>

        <repository>
          <id>github</id>
          <url>https://maven.pkg.github.com/u2waremanager/i
o.u2ware.common.u2ware-common-docs</url>
          <snapshots>
            <enabled>true</enabled>
          </snapshots>
        </repository>

      </repositories>
    </profile>
  </profiles>

  <servers>
    <server>
      <id>github</id>
      <username>username</username>
      <password>token</password>
    </server>
  </servers>
</settings>
```

- username과 password에 각각 자신의 github username과 token값을 입력해줍니다.

- C:\Users\user\.m2\repository\io\u2ware\common 로 이동합니다.

  아래 사진의 4개의 폴더를 지워줍니다.

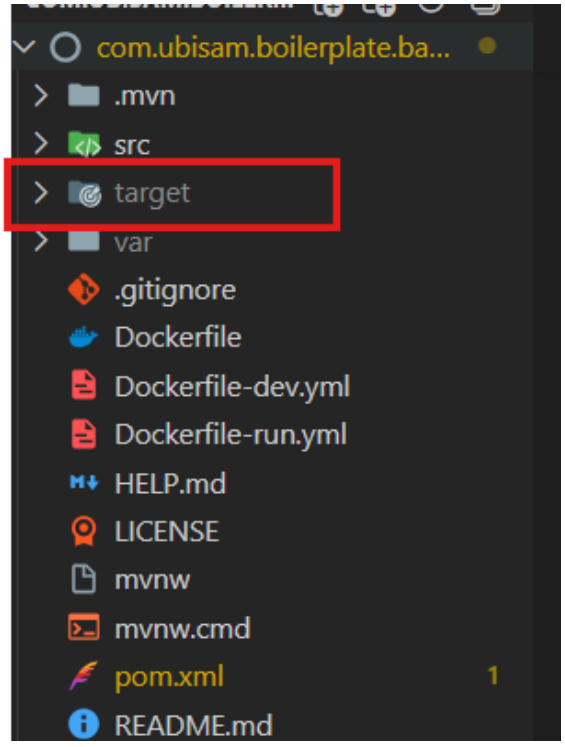
u2ware-common-data	2026-03-05 오후 5:37
u2ware-common-docs	2026-03-05 오후 5:37
u2ware-common-oauth2	2026-03-24 오후 3:46
u2ware-common-stomp	2026-03-05 오후 5:37

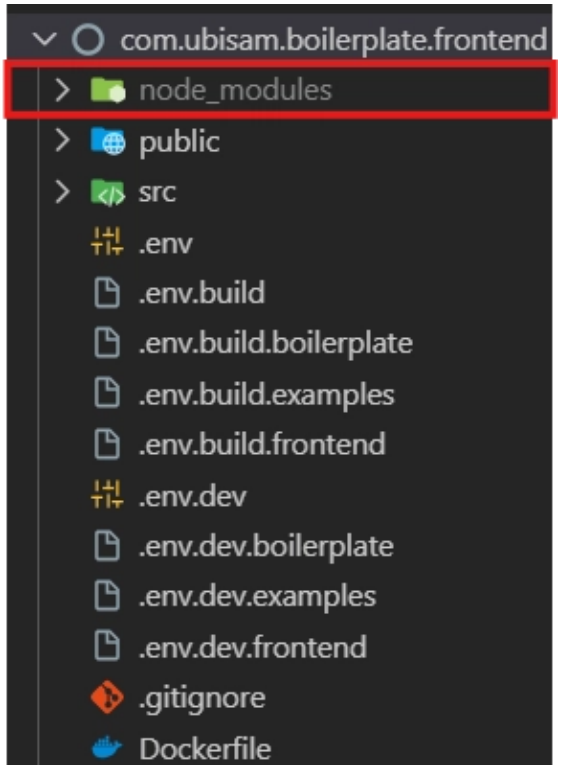
- 다시 Boilerplate Workspace를 엽니다.

  Backend 코드 기준 target 폴더, Frontend 코드 기준 node_modules 폴더를 지워줍니다.

  없으면 안지우셔도 됩니다.

  - 명령어가 알아서 지우고 다시 설치하지만 개인적으로 수동으로 지우는걸 추천드립니다.





- Backend 터미널에서는 ./mvnw clean install 혹은 .\mvnw clean install 을

  Frontend 터미널에서는 npm install 을 해줍니다.

  💡 .\mvnw clean install

💡 npm install

```
PS C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.backend> .\mvnw clean install
```

```
PS C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.frontend> npm install
```

- Backend 코드는 아래 사진과 같이 뜨면 정상적으로 설치된 것입니다.

```
[INFO] Results:
[INFO]
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ examples-backend ---
[INFO] Building jar: C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.backend\target\examples-backend-3.5.11.jar
[INFO]
[INFO] --- spring-boot:3.5.11:repackage (repackage) @ examples-backend ---
[INFO] Replacing main artifact C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.backend\target\examples-backend-3.5.11.jar with repackaged arch
[INFO] The original artifact has been renamed to C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.backend\target\examples-backend-3.5.11.jar.or
[INFO]
[INFO] --- install:3.1.4:install (default-install) @ examples-backend ---
[INFO] Installing C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.backend\pom.xml to C:\Users\user\.m2\repository\com\ubisam\boilerplate\examp
[INFO] Installing C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.backend\target\examples-backend-3.5.11.jar to C:\Users\user\.m2\repository\c
.5.11.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  25.908 s
[INFO] Finished at: 2026-03-25T11:20:13+09:00
[INFO] ------------------------------------------------------------------------
PS C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.backend>
```

- Frontend 코드는 아래 사진과 같이 뜨면 정상적으로 설치된 것입니다.

```
PS C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.frontend> npm install

up to date, audited 374 packages in 2s

112 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\user\Desktop\CHJ\workspace\semina\com.ubisam.boilerplate.frontend>
```

- 그 후 다시 빌드를 해줍니다.

이렇게 3월 24일 세미나에 대한 내용 공유였습니다.

제 소스 파일은 GitHub에 올려 두었습니다.

https://github.com/ubisam-heung/com.ubisam.projects.semina.backend

https://github.com/ubisam-heung/com.ubisam.projects.semina.frontend

긴 글이었는데도 읽어 주셔서 감사합니다. 🙂